数据库原理

Database Principle

陶宏才、李晓航

西南交通大学

计算机与人工智能学院

第三章 关系数据库系统

学习目的和要求

- 关系数据模型
- 关系上的完整性约束或限制
- 关系代数



3.1 关系数据模型

回答如下问题:

1. 如何表达data本身?
2. 如何表达数据间的联系?
3. 能表达哪些完整性约束?
4. 完整性约束如何设定?
5. 完整性约束的效果?
6. 如何进行关系模型的DB设计?



一.SQL语言简介

最早在IBM System-R 开发的查询语言；由于其广泛的使用，出现标准化需求，形成SQL标准。

第一个SQL标准由ANSI于1986年制订，称为SQL-86；1989年作了些许改进，称为SQL-89；1992年由ANSI和ISO合作，作了较大改动，称为SQL-92(SQL2)，这是目前大多数商用RDBMS支持的版本；1999年提出SQL:1999(SQL3)，是SQL-92的扩展。

关系模型中的关系，利用SQL-92来定义和操纵；SQL-92标准中用“表”（Table）代表“关系”（Relation）。

关系 →

{ $<a_{11}$	, a_{12}	, a_{13}	, ……	, $a_{1n}>$,
$<a_{21}$	, a_{22}	, a_{23}	, ……	, $a_{2n}>$,
$<a_{31}$	, a_{32}	, a_{33}	, ……	, $a_{3n}>$,
……	……	……	……	……
……	……	……	……	……
$<a_{m1}$	, a_{m2}	, a_{m3}	, ……	, $a_{mn}>$ }

→ 表

SQL中用于创建(Create)、删除(Drop)和修改(Alter)“表结构”的部分叫DDL，而对“表中数据”进行插入(Insert)、删除(Delete)、修改(Update)和查询（Select）的部分叫DML；

归结起来，SQL语言DDL和DML命令为：

1. SQL DDL——针对表结构
   (1) Create
   (2) Drop
   (3) Alter
2. SQL DML——针对表中数据
   (1) Insert
   (2) Delete
   (3) Update
   (4) Select

二.关系模型

1. 关系（Relation）

概念：用于描述数据本身、数据之间联系。俗称“表”。

示例：一张“学生信息表”（数据本身）、“学生选课表”（数据间的联系）。

表构成：由“列”（Column）和“行”（Row）组成。各列构成“关系模式（Schema）”，各行构成“关系实例（Instance）”。

列：有时也称“字段”（Field）、“属性”“成员”,表示为：A_i。

行：有时也称“元组”（Tuple）、“记录”（Record）。表示为：$t=<a_1, a_2, \ldots, a_n>$，其中$a_i$为对应属性$A_i$的值。

关系模式：由关系名、各个列构成。

表示为：R（$A_1, A_2, \ldots, A_n$）。

域（Domain）：列或属性的取值范围。

关系实例：记录集或元组集，常简称关系；行序不重要，而列序重要(关系代数中有时用列号来代表列名)；每个元组的字段必须对应关系模式中的字段。

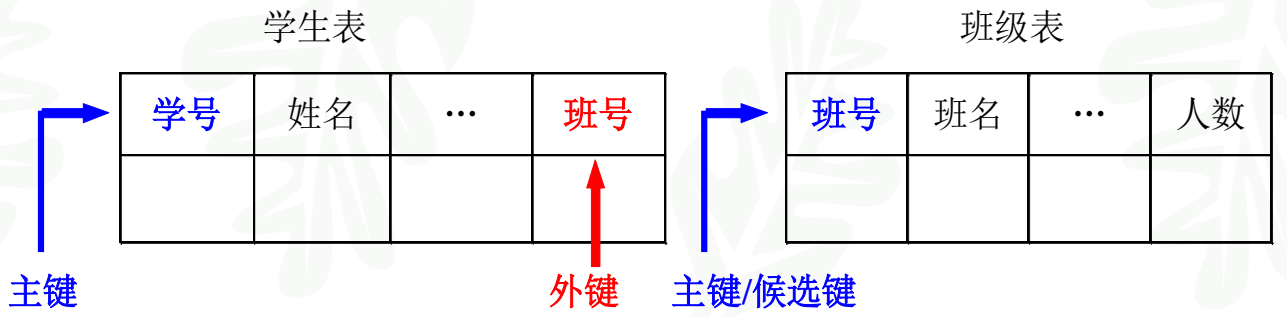
关系实例表示：$\{<a_1, a_2, \ldots, a_n>\}$，$a_i$为属性$A_i$的值。

候选键：能唯一识别元组的最小字段集（可能有多个）。

主键（Primary Key，PK）：一个唯一识别关系实例的最小字段集合。一个关系只能有一个主键。

外键（Foreign Key，FK）：一张表中的某个（组）字段是另一张表中的候选键/主键。

示例：




关系数据模型
结构
对象
对象间联系
表
操作
Insert/Delete/Update/Select
约束
?

2. 关系模型完整性限制（Integrity Constraints, ICs）

目的：RDBMS必须帮助阻止不合规数据的输入。

概念：要求存入DB的数据应满足的条件。

ICs指定的时间：定义完一个DB模式时/后。

ICs何时起作用：当表中数据发生变化（或称更新，由Insert、Delete、Update操作导致的）时，DBMS即检查更新的数据是否满足IC指定的条件。

可被指定的ICs：域限制(域约束)、主键限制、唯一限制、外键限制、一般性限制等。用SQL的DDL子语言可定义这些限制，如果某限制被违背，系统将返回限制名，并用来识别错误。

（1）域限制（Domain Constraint）

概念：指数据类型的约束，是关系模型中最基本的约束。

（2）主键限制（Primary Key Constraint）

概念：针对主键而言，保证主键的完整性。

要求：主键值必须唯一，且不能为空值（即非空）。

（3）唯一限制（Unique Constraint）

概念：针对候选键而言，保证候选键的完整性。

要求：值唯一，允许有一个且仅一个空值。

（4）外键限制（Foreign Key Constraint）

目的：用来维护表间的数据一致性，即其中一张表的变动，可能要求另一张表要作出某些变动，以保持数据一致。为使DBMS能做这样的检查，则应指定这种涉及两个表的ICs，此即外键限制。

如何维护两张表数据一致？

主表更新（Insert/Delete/Update），从表？

主键

外键

班级表——主表

班号	班级名	…
rj0001	软件1班	
~~rj0002~~	~~软件2班~~	
zm0001	智茅1班	
zm0002	智茅2班	
rj0003	软件3班	

学生表——从表

学号	姓名	…	班号
00001	张三		rj0001
00002	李四		rj0001
~~00031~~	~~张三~~		~~rj0002~~
~~00055~~	~~王五~~		~~rj0002~~

主表

- Insert ✓
- Delete → SQL Server实现
- Update → SQL Server实现

SQL Server实现

- Set Null
- Set Default
- Cascade
- No Action

SQL-92推荐的动作

小结：主表→从表的一致性维护



从表更新（Insert/Delete/Update），如何保证两张表数据一致？

学生表——从表

学号	姓名	…	班号
00001	张三		rj0001
~~00002~~	~~李四~~		~~rj0001~~
00031	张三		rj0002
00055	王五		rj0002
00066	陆六		?

References (参照)

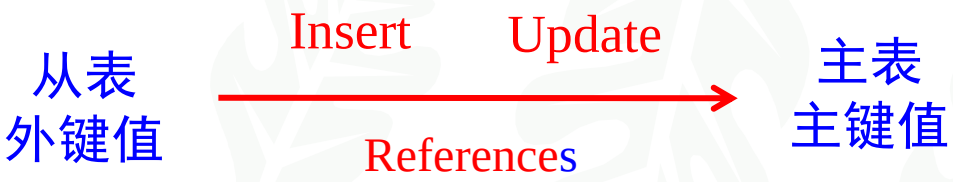
References (参照)

班级表——主表

班号	班级名	…
rj0001	软件1班	
rj0002	软件2班	
zm0001	智茅1班	
zm0002	智茅2班	

从表
- Insert ✓
- Update ✓
- Delete ✓

小结：从表→主表的一致性维护



从表：指含有外键的表。

主表：指另一张表的外键在此作主/候选键的表。

两个通过外键联系的表的三种操作对完整性的影响：

① 对主表中的主键做操作

★插入：要求插入值满足主键限制即可，不影响其它表。

★修改：可能会影响与该主键相关的从表的外键值。如影响从表的外键值，有两个行动(SQL Server)可保证两张表的一致：一是Cascade，级联修改对应从表的所有外键值，使之与主键一致；其二是No Action，不允许修改主表中的主键值。

★删除：可能会影响与该主键相关的从表的外键值。如影响从表的外键值，行动有二：一是No Action，不允许删除主表的主键值；二是Cascade，级联删除从表中相应外键值所在的行。

② 对从表中的外键操作

★插入：要求插入的外键值应“参照”（References）主表中的主键值。

★修改：要求修改的外键值“参照”主表中的主键值。

★删除：不需要参照主表中的主键值。

表间数据的完整性：由上所述，维护表间完整性实际是从两个方向上完成，即：

① 主表 → 从表：表示“主表”中的主键值在修改和删除时，“从表”中与该主键值相同的外键值可“级联”（CASCADE）修改和删除，或“禁止”（NO ACTION）“主表”主键值的修改和删除。



② 从表 → 主表：表示“从表”中的外键值在插入和修改时，其值应“参照”（References）“主表”中的主键值。

表间数据完整性维护的两种实现方法：

① 对从表定义外键限制（或称参照完整性）完成主表和从表间两个方向的数据完整性；

② 利用触发器完成维护两表间数据完整性，即主表的触发器维护主表到从表方向的数据完整性，而从表的触发器维护从表到主表方向的参照完整性。

外键限制定义的条件：

①定义外键限制的列必须是另一个表中的主键或候选键；

②在含外键的表上定义

（5）一般性限制（General Constraints）

主键限制、唯一限制和外键限制分别是维护表中3种特殊字段（主键、候选键和外键）的完整性，对表中其他一般性的字段的完整性，则用一般性限制来维护。

一般性限制实现的两种方式：表限制和断言（Assertion）。

表限制：与单个表有关，SQL Server中称检查限制（Check Constraint）。

断言：与多个表有关（注：SqlServer不支持断言）。

3. 完整性限制（ICs）何时起作用

关系创建并指定了ICs后，当关系“更新”时应实施检查。

(1)对域限制、主键限制、唯一限制和一般性限制的实施

域限制/主键限制/唯一限制在SQL语句执行时立即检查，只要插入/删除/修改命令违背了这几个限制，即被拒绝。而一般性限制的违背检查通常是在每个SQL语句之后。

(2) 对外键限制的实施

从表到主表方向的完整性立即检查；而主表到从表方向，在检查出对从表有影响时，需要实施相应行动来完成。


关系数据模型
结构
对象
对象间联系
表
操作
Insert/Delete/Update/Select
约束
域限制
主键限制
唯一限制
外键限制
一般性限制

4. SQL Server对完整性限制的支持

SQL Server中的数据完整性可粗分为两大类，如下表。

完整性分类	现有实现方式	备 注
表内的完整性	Default（缺省）	指定列（或域）的缺省值
	Rule（规则）	指定列（或域）的取值范围
	Check Constraint（检查限制）	均有列级（即只涉及一列）和表级（涉及表中多列）
	Primary Key（主键限制）	
	Unique（惟一限制）	
表间的完整性	Foreign Key（外键限制）或 References(参照限制)	亦分列级与表级
	Trigger（触发器）	可利用触发器来维护表间数据完整性

5. SQL Server完整性限制的设定

（1）Default

概念：用于为列或用户自定义数据类型指定缺省值，每一列或自定义类型只能有一个缺省值。

设定方式：

对象类型，如Table, Database, View, Default, Rule, Index, Trigger, Function, Procedure。

① 表定义时设定

圆括号　表名　定长字符类型　非空　逗号　缺省值

示例：

```
CREATE TABLE publishers
(pub_id     char(4)       NOT NULL,
 pub_name   varchar(40)   NULL,
 city       varchar(20)   DEFAULT 'Pasadena',
 state      char(2)       DEFAULT 'CA')
```

Variable, 可变长度/不定长字符类型

② 创建Default

当多个表中的列其缺省值相同时，这种方式很有用。

步骤：

(A) CREATE DEFAULT 缺省名 AS 缺省值

(B) sp_bindefault '缺省名', '表名.列名'

sp_，系统存储过程

示例：

```
CREATE DEFAULT dft_state AS 'CA'
sp_bindefault 'dft_state', 'publishers.state'
```

注意：要求列的类型与缺省值相同。绑定了缺省值后，并不会对绑定缺省值之前表中已存在的值产生影响，而只对绑定之后的值产生影响。

取消某列的缺省：sp_unbindefault '表名.列名'

删除缺省：DROP DEFAULT 缺省名

注意：应保证该缺省已从所有绑定的列上摘除，否则删除不会成功。

（2）Rule

概念：针对表中的某一列，指明某列的取值范围。在更新该列值时,RDBMS首先要检查是否在该规则规定的范围内。

设定步骤：

① CREATE RULE 规则名 AS 规则

示例：

CREATE RULE state_rule AS @state IN('CA','CO','WA')

局部变量(用户定义变量)
@@开头的变量为系统变量

注意：

枚举值

字符匹配：_、%、[ ]、[^]、$

★规则可用IN(...)，BETWEEN ... AND ...，关系式 < 、 > 、<=、>=、<>、=等和LIKE等操作符描述。

★创建规则时，应注意AS后有一个以@开头的局部变量。

② sp_bindrule 规则名，'表名.列名'

示例：sp_bindrule state_rule, 'publishers.state'

解除规则绑定：sp_unbindrule ‘表名.列名’

删除规则：DROP RULE 规则名

用sp_addtype定义

扩展用法：将创建好的缺省和规则绑定到用户定义类型上

sp_bindrule 规则名，用户定义类型

sp_bindefault 缺省名，用户定义类型

要求：先应创建好用户定义类型，然后再绑定。

将缺省和规则从用户定义类型摘除：

sp_unbindefault 用户定义类型

sp_unbindrule 用户定义类型

查看创建规则和缺省的过程：

sp_helptext 规则名或缺省名

注意：①缺省必须从属于规则，缺省与规则都有时，必须同时满足；②一般在绑定一个新规则或缺省时，应先摘除旧规则或旧缺省，如没有摘除，则自动用新规则/新缺省替换旧的。

（3）检查限制

概念：类似于规则，要求用户插入到列或表中的数据满足限制条件。

分类：分列级和表级两种。 列级针对表中一列，表级则针对同一表中多列。

列级检查限制(用IN、BETWEEN...AND或LIKE表达)：

```
CREATE TABLE  publishers
( pub_id       char(4)          NOT NULL,
 CONSTRAINT  pub_id_constraint
 CHECK( pub_id IN ('234','3344','564')
                  OR pub_id LIKE '43[0-9][0-9]'),
 city          varchar(20)      NULL,
 state         char(2)          NULL)
```

定义约束名（限制名）。
如不定义，系统则自动定义。

逻辑运算：与And，或Or，非Not

其中：pub_id_constraint为限制名。

表级检查限制：

```
CREATE TABLE  discounts
   ( discounttype        varchar(40)    NOT NULL,
    store_id             char(4)        NULL,
    lowqty               smallint       NULL,
    highqty              smallint       NULL,
    discount             float          NOT NULL,
   CONSTRAINT low_high_check
   CHECK (lowqty <= highqty))
```

整型：int，4 bytes；
小整型：smallint，2 bytes；
微整型：tinyint，1 byte。

注意：

缺省值须满足检查限制要求。

（4）主键限制

概念：不能出现空值，且所有值惟一。在定义了主键限制后，系统自动为该表以主键生成一个聚簇（Clustered）索引。

分类：分列级和表级两种。 列级针对表中一列，表级则针对同一表中多列。

列级主键限制：

```
CREATE TABLE  publishers
( pub_id        char(4)          PRIMARY KEY,
  pub_name      char(30),
  city          varchar(20)      NULL,
  state         char(2)          NULL)
```

关于索引(Index)

索引：加快查询速度。

升序(**Asc**ending)——ASC
降序(**Desc**ending)——DESC

非聚簇索引(Nonclustered index)

Index1

学号	Pointer
x00033	
x00088	
x00199	
x00277	

Index2

姓名	Pointer
LS	
SS	
WW	
ZS	

Index3

学号	姓名	Pointer
x00033	SS	
x00088	LS	
x00199	ZS	
x00277	WW	

学生表

学号	姓名	…
x00199	ZS	
x00088	LS	
x00277	WW	
x00033	SS	

表级主键限制：

```
CREATE TABLE sales
( stor_id        char(4)        NOT NULL,
  date           datetime       NOT NULL,
  ord_num        varchar(20)    NOT NULL,
   CONSTRAINT pk_sales_constr
   PRIMARY KEY NONCLUSTERED (stor_id, ord_num))
```

下面写法对吗？

```
CREATE TABLE sales
( stor_id        char(4)        PRIMARY KEY,
  ord_num        varchar(20)    PRIMARY KEY,
  date           datetime       NOT NULL )
```

（5）唯一限制

概念：主要是针对候选键的限制。在定义了惟一限制后，系统自动为该表以候选键生成一个非聚簇索引，当然在定义时可改成聚簇索引。

与主键限制之区别：所有值惟一，最多只能有一个空值。默认索引为非聚簇（NonClustered）索引。

分类：分列级和表级两种。列级针对表中一列，表级则针对同一表中多列。

列级唯一限制：

```
CREATE TABLE publishers
( pub_id        char(4)        UNIQUE,
  pub_name      char(30))
```

表级唯一限制：

```
CREATE TABLE sales
( stor_id        char(4)        NOT NULL,
  ord_num        varchar(20)    NOT NULL,
  date           datetime       NOT NULL,
  CONSTRAINT uq_sales_constr
  UNIQUE CLUSTERED (stor_id, ord_num))
```

仅就代码而言，下面写法对吗？

```
CREATE TABLE sales
( stor_id        char(4)        UNIQUE,
  ord_num        varchar(20)    UNIQUE,
  date           datetime       NOT NULL)
```

再问：与前一语句唯一限制的含义相同吗？

（6）外键限制

复习：维护表间完整性实际是从两个方向上完成，即：



外键限制定义的条件：

①定义外键限制的列必须是另一个表中的主键或候选键；

②在含外键的表上定义

外键限制分类：分列级和表级两种。 列级针对表中一列，表级则针对同一表中多列。

列级外键限制：

```
CREATE TABLE titles
( title_id        char(6)          NOT NULL,
  title           varchar(4)       NULL,
  pub_id          char(4)          NULL
  CONSTRAINT      pub_id_const
  REFERENCES      publishers (pub_id)
  ON DELETE NO ACTION
  ON UPDATE CASCADE,
  notes           varchar(23)      NULL)
```

从表→主表方向完整性

主表→从表方向完整性

说明：No Action是系统默认行动，故No Action行可隐去不写；

表级外键限制：

```
CREATE TABLE salesdetail
( stor_id          char(4)          NOT NULL,
  ord_num          varchar(20)      NOT NULL,
  title_id         tid              NOT NULL,
  qty              smallint         NOT NULL,
  discount         float            NOT NULL,
  CONSTRAINT       sales_constr
FOREIGN KEY (stor_id, ord_num)
REFERENCES sales (stor_id, ord_num)
ON DELETE NO ACTION
ON UPDATE CASCADE ,

CONSTRAINT  titles_constr
FOREIGN KEY (title_id)
REFERENCES titles(title_id)
ON DELETE CASCADE
ON UPDATE CASCADE )
```

外键(Stor_id, ord_num)的限制

外键(title_id)的限制



注意：

① 外键限制定义中的行动，表示当对主表的主键值做某种操作（删除或修改时），可采取的行动。有二种可选择：

（A） CASCADE（针对从表的行动）；

（B） NO ACTION（针对主表的行动）；

说明：以上二种行动是MS SQL SERVER及SYBASE中支持的，而在SQL-92标准中定义有4种行动，分别是：Cascade、No Action、Set Null及Set Default。由此也说明，具体的数据库产品不一定完全支持SQL标准。

② 如果不定义行动，则缺省处理为NO ACTION。

（7）触发器（Trigger）

概念：如前所述，利用主表和从表的触发器，也可以保持主/从表间的数据完整一致性。

示例：publishers表的主键是pub_id，它也出现在titles表中，即titles表中的pub_id是外键，也就是说，publishers为主表，而titles则为从表。要维护两表间的完整性，可对titles用外键限制或参照限制来维护它们间的完整性，或对publishers和titles用触发器来维护它们间的完整性。

触发器的创建：

```
CREATE TRIGGER 触发器名
ON 表名|视图名
{FOR|AFTER|INSTEAD OF} {[INSERT][,][UPDATE][,][DELETE]}
AS
    SQL语句
RETURN
```

说明：

①触发器不带参数，不被用户和程序调用，只能由用户对DB中的表/视图进行操作时触发(即：由操作激发)。

②激发触发器的三种操作，即：插入、修改和删除。

③一旦某操作激活了某个触发器，系统就会将该操作与该操作激活的触发器作为一个事务提交（Commit）或回退/回滚/滚回（Rollback）。

④触发器分2种，即AFTER（只能在表上创建）和INSTEAD OF（表或视图）。FOR是为与早期版本兼容而保留，默认为AFTER类型。AFTER触发器在操作执行后才执行；INSTEAD OF触发器则在操作之前并代替操作的活动。

⑤一个触发器同时含有两张表，即：inserted和deleted。当操作为插入时，则新数据也会写入inserted表中；当操作为删除时，删除的数据（即：老数据）会保存在deleted表中；而当操作为修改时，会同时用到这两张表，先删后写。

由删除操作激发的主表删除触发器示例：

```
CREATE  TRIGGER      pub_del
ON  publishers
FOR  DELETE
AS
        IF @@rowcount=0
                RETURN
        DELETE titles
        FROM  titles t, deleted d
        WHERE t.pubid=d.pubid
  RETURN
```

模拟级联删除



由修改操作激发的主表修改触发器示例：

```
CREATE TRIGGER   pub_update
ON publishers FOR UPDATE   AS
      declare @num_rows int
      select @num_rows=@@rowcount
      IF @num_rows=0   RETURN
      IF update(pubid)
      begin
            if @num_rows > 1
            begin
                raiserror (‘不支持主键的多行更新！’,8,1)--注意版本区别
                rollback   transaction
                return
            end

            UPDATE titles
            SET pubid = i.pubid
            FROM  titles t, deleted d, inserted i
            WHERE t.pubid=d.pubid
   end
   RETURN
```

定义变量

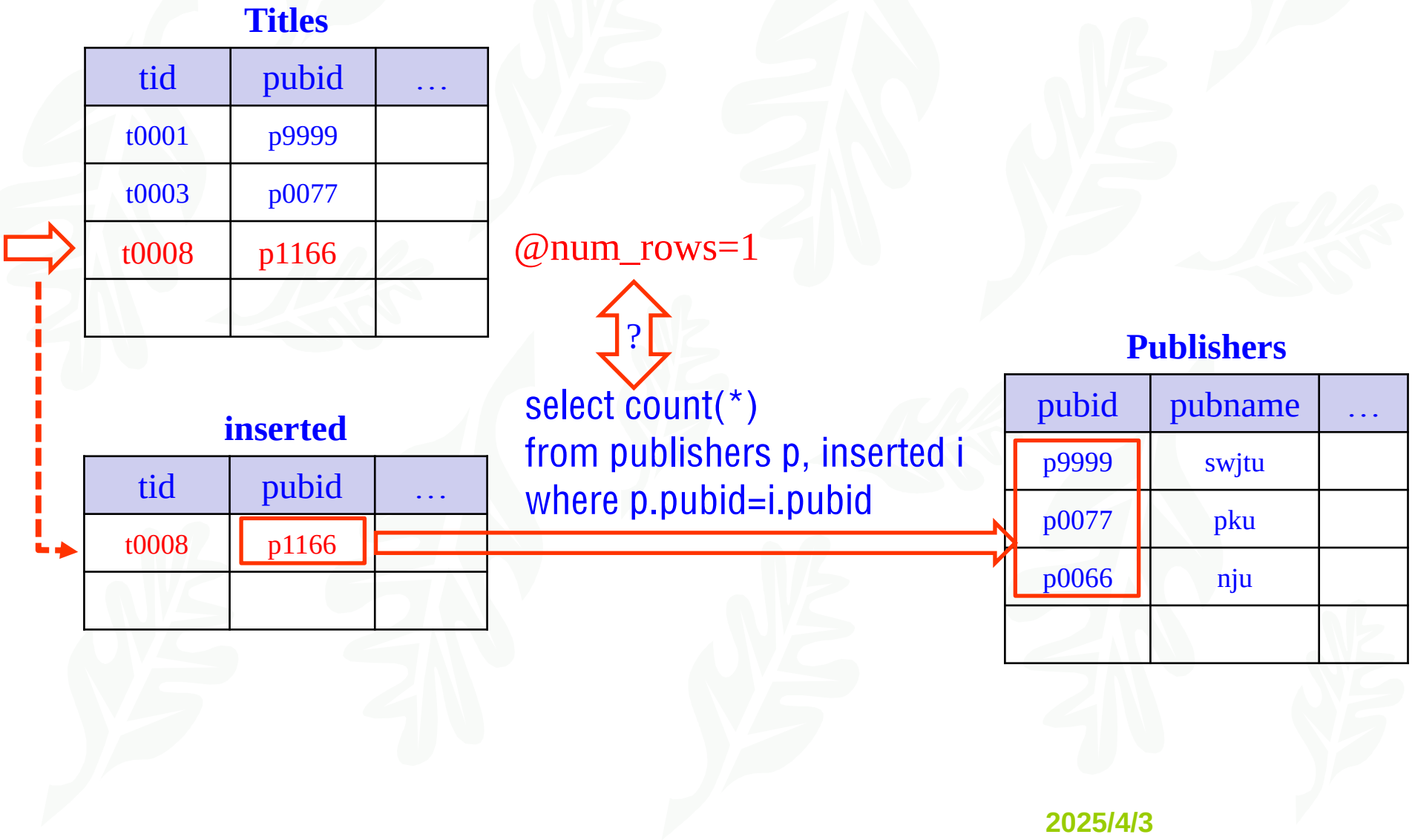
给变量赋值

模拟级联修改

由插入和修改操作激发的从表插入修改触发器示例：

```
CREATE TRIGGER   title_iu
ON titles FOR INSERT, UPDATE   AS
    declare @num_rows int
    select @num_rows=@@rowcount
    IF @num_rows=0   RETURN
    IF (select count(*)
        from publishers p, inserted I
        where p.pubid=i.pubid) != @num_rows
    begin
        raiserror (‘试图插入/更新非法pub_id到titles表中！’,8,1)
        --注意raiserror语法格式因SQL Server版本不同带来的变化
        rollback  transaction
        return
  end
  RETURN
```

IF (select count(*)
from publishers p, inserted I
where p.pubid=i.pubid) != @num_rows的解析



查看触发器的创建过程：

sp_helptext 触发器名

查看表依赖的触发器或触发器涉及的表：

sp_depends 表名或触发器名

删除触发器：

DROP TRIGGER 触发器名

注意：

① 不能对临时表创建触发器；

② 如果建立触发器的表被删除，那么其上的触发器将被自动删除。

外键限制与触发器在维护表间数据一致性上的区别：

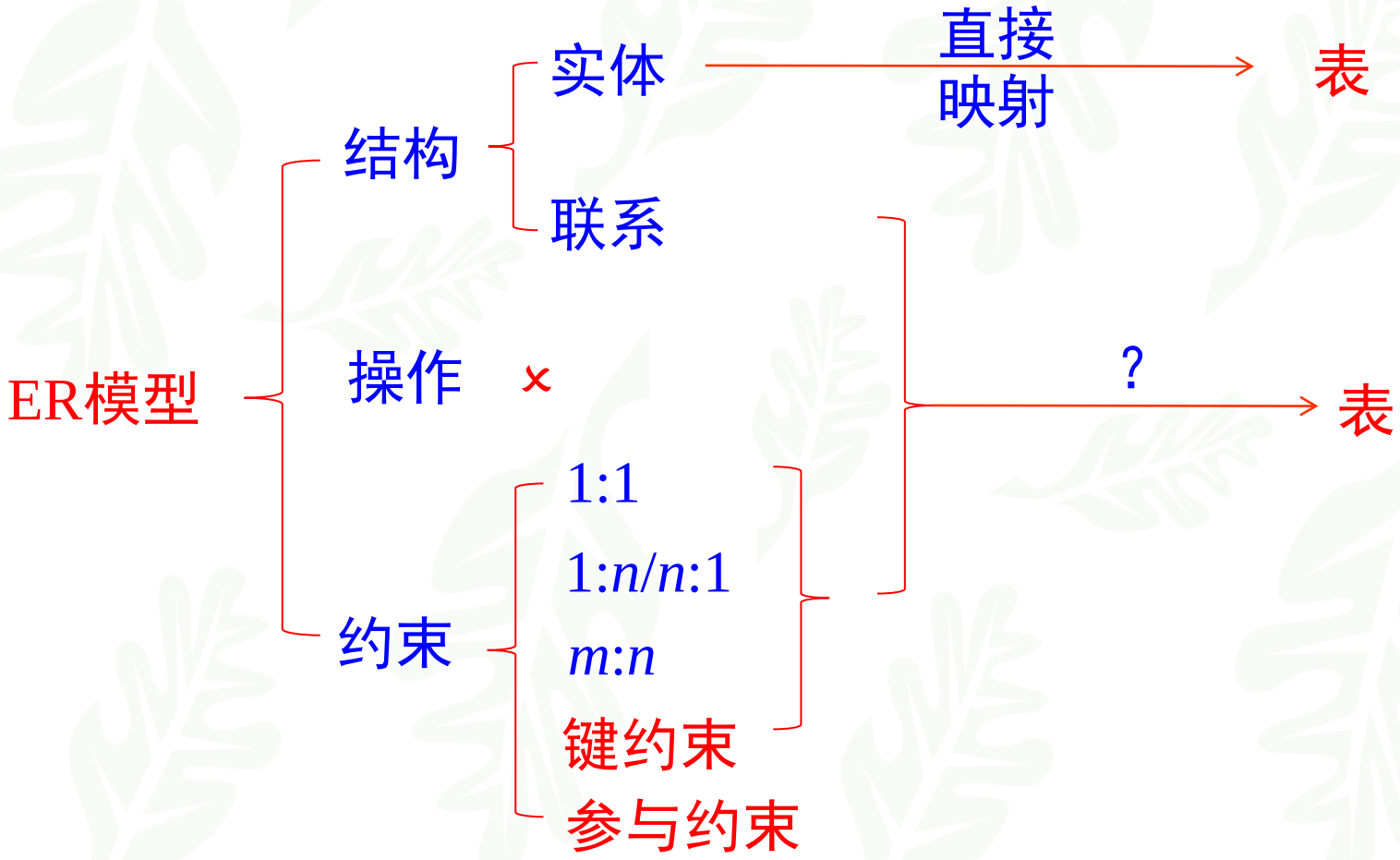
① 外键限制有前提条件，而触发器没有。

② 外键限制只需在从表定义，而触发器则在主表和从表都要写。

③ 外键限制的行动固定，而触发器的行动灵活可定义。

6. Logical DB Design: ER to Relational

ER → Relation的转换是将ER图转换成带有相关限制的表的集合。



（1）Entity Sets to Tables

映射直接，实体型中的每个属性对应表的列。实体的主键即为表的主键。

（2）Relationship Sets 的转换

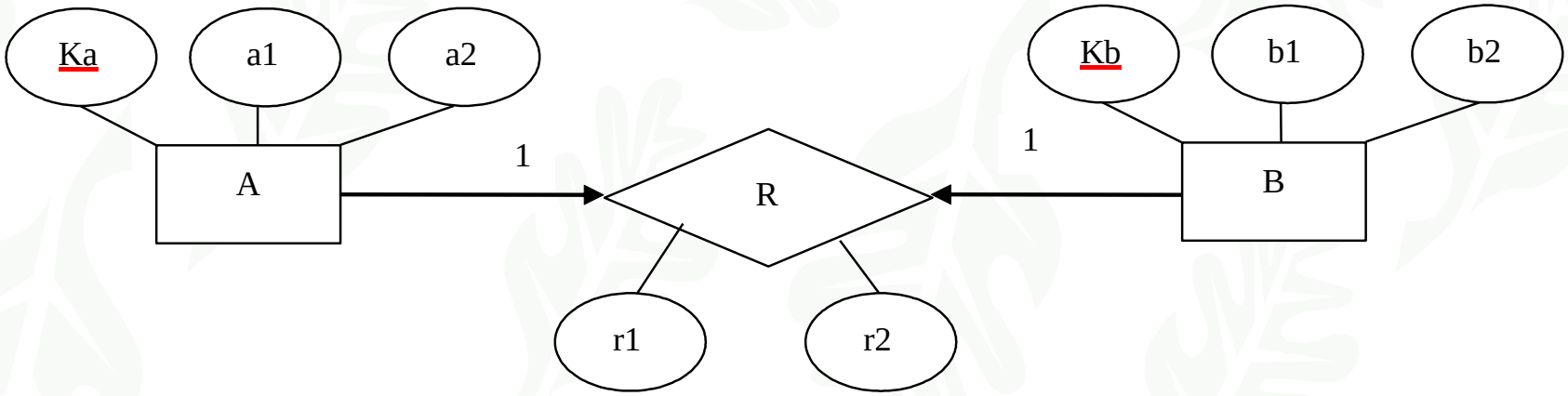
一般，联系型的转换方法有两种，一种是将联系型转换为表；另一种则是联系型不转换为表，但应将其描述属性移动到带有键约束的实体转换的表中（也就是说，如果不存在键约束，则联系型必须转换为表，即此时只有一种转换方法）。

（3）联系型转换为表

由于联系与实体关联，故转换后所得表的列组成为：各参与实体的主键（联系的隐含属性）、联系型的描述属性。

下面，将按1:1、1:n、m:n三种情况说明其转换

1）1:1联系



转换结果为：

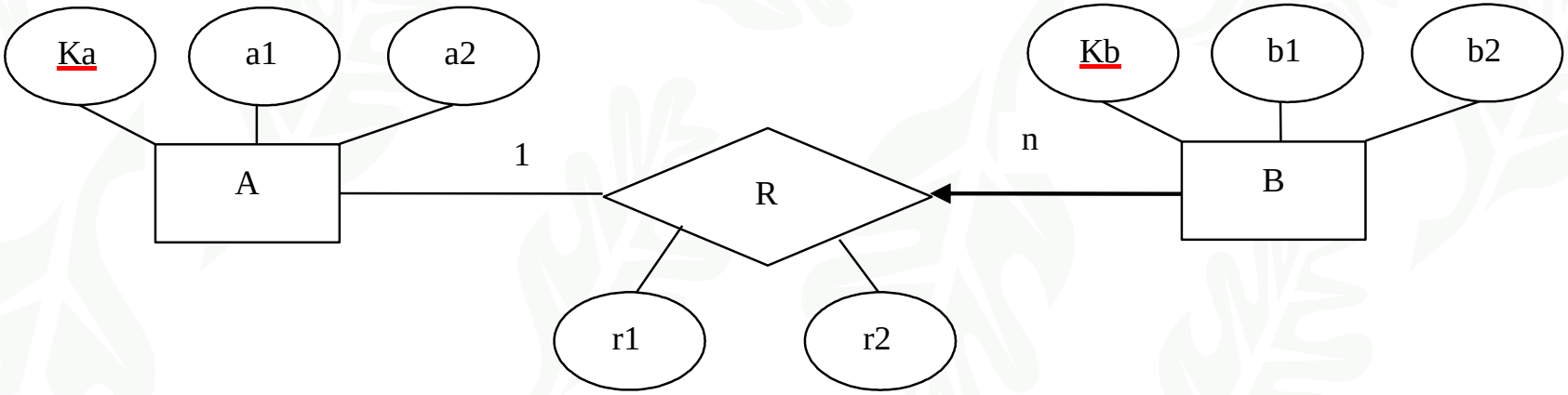
A (Ka,a1,a2), Ka为主键

B (Kb,b1,b2), Kb为主键

R (Ka, Kb, r1, r2), Ka, Kb均为外键

由于有两个键约束，故Ka, Kb都能单独作为R的主键。如果指定Ka为主键，则Kb为候选键；否则，如指定Kb为主键，则Ka为候选键。

2）1:n联系



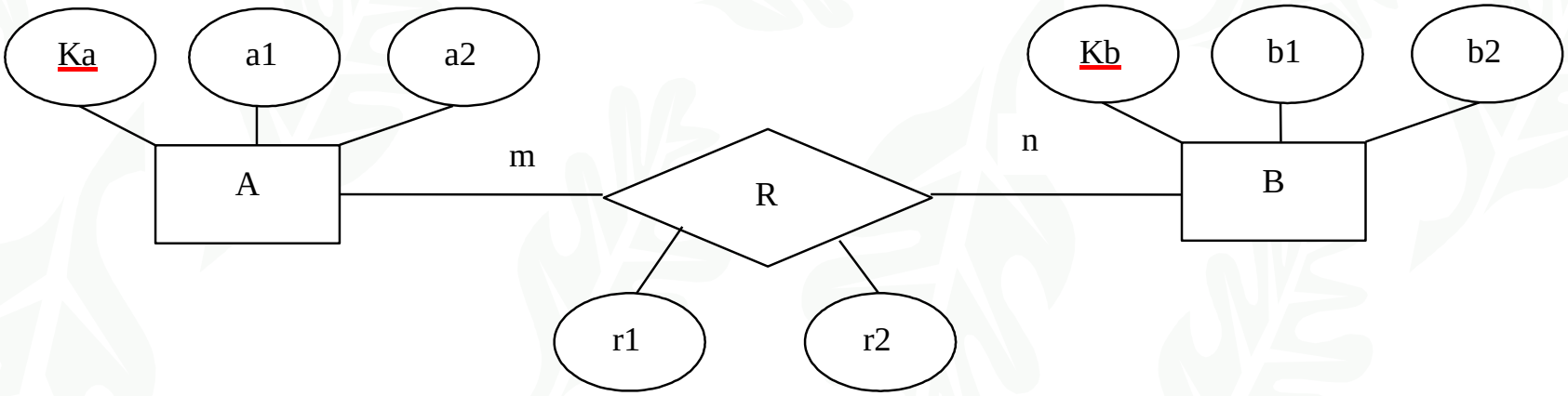
转换结果为：

A (Ka,a1,a2), Ka为主键

B (Kb,b1,b2), Kb为主键

R (Kb, Ka, r1, r2), Kb为主键，另外，Ka, Kb均为外键

3）m:n联系



转换结果为：

A (Ka,a1,a2), Ka为主键

B (Kb,b1,b2), Kb为主键

R (Ka, Kb, r1, r2), Ka和Kb组合为主键，另外，Ka, Kb均为外键

（4）带键约束的联系型可不转换为表

如果联系型不转换为表，由于联系型可能带有描述属性，另外从联系可转换成表来看它还包括有参与实体的主键这样的属性，因此必须将其描述属性移动到有键约束的实体中，同时拷贝另一参与实体的主键到带有键约束实体所转换的表中。

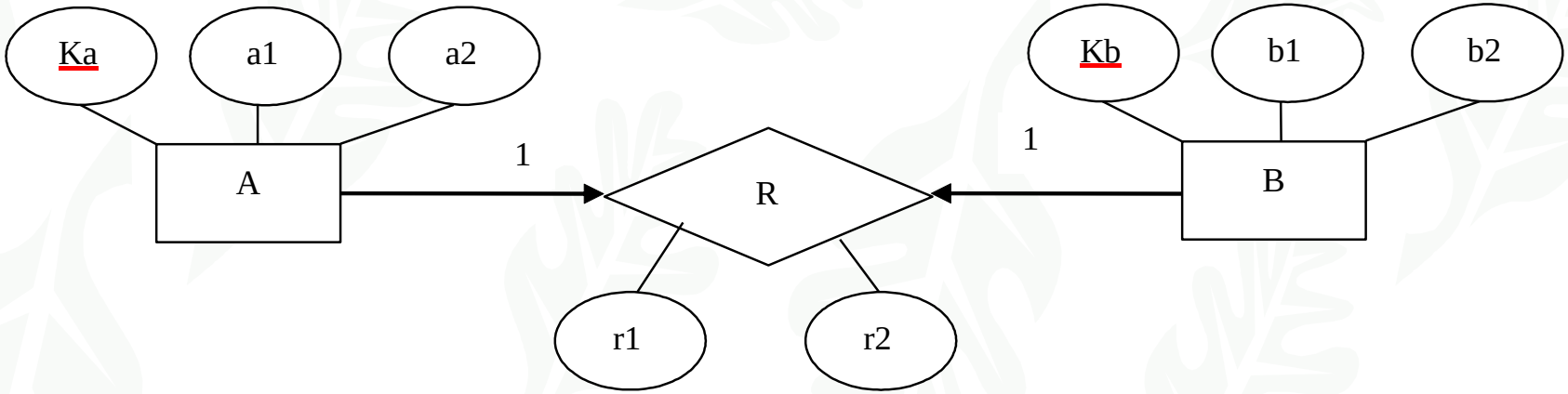
键约束只存在于1:1和1:n联系中，因此只有这两种联系才可使用不转换的方法。

再次强调：对于m:n联系，一般必须将其转换为表。

对于1:1联系，由于存在两个键约束，因此联系的属性既可移动到A实体所转换的表中，也可以移动到B实体所转换的表中；但对于1:n联系，联系的属性必须移动到n方（带键约束）实体所转换的表中。

下面，将按1:1、1:n二种情况来说明其转换

1）1:1联系



结果一（将描述属性移动到A实体所转换的表中，同时将Kb拷贝过去）：

A (Ka, a1, a2, r1, r2, Kb), Ka为主键, Kb为外键

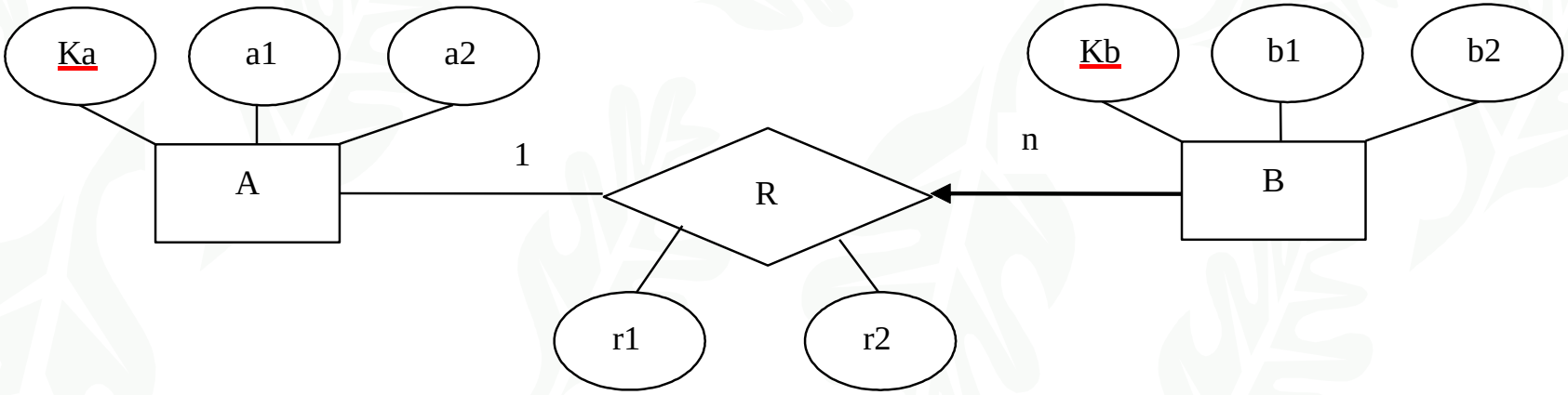
B (Kb, b1, b2), Kb为主键

结果二（将描述属性移动到B实体所转换的表中，同时将Ka拷贝过去）：

A (Ka, a1, a2), Ka为主键

B (Kb, b1, b2, r1, r2, Ka), Kb为主键, Ka为外键

2）1:n联系

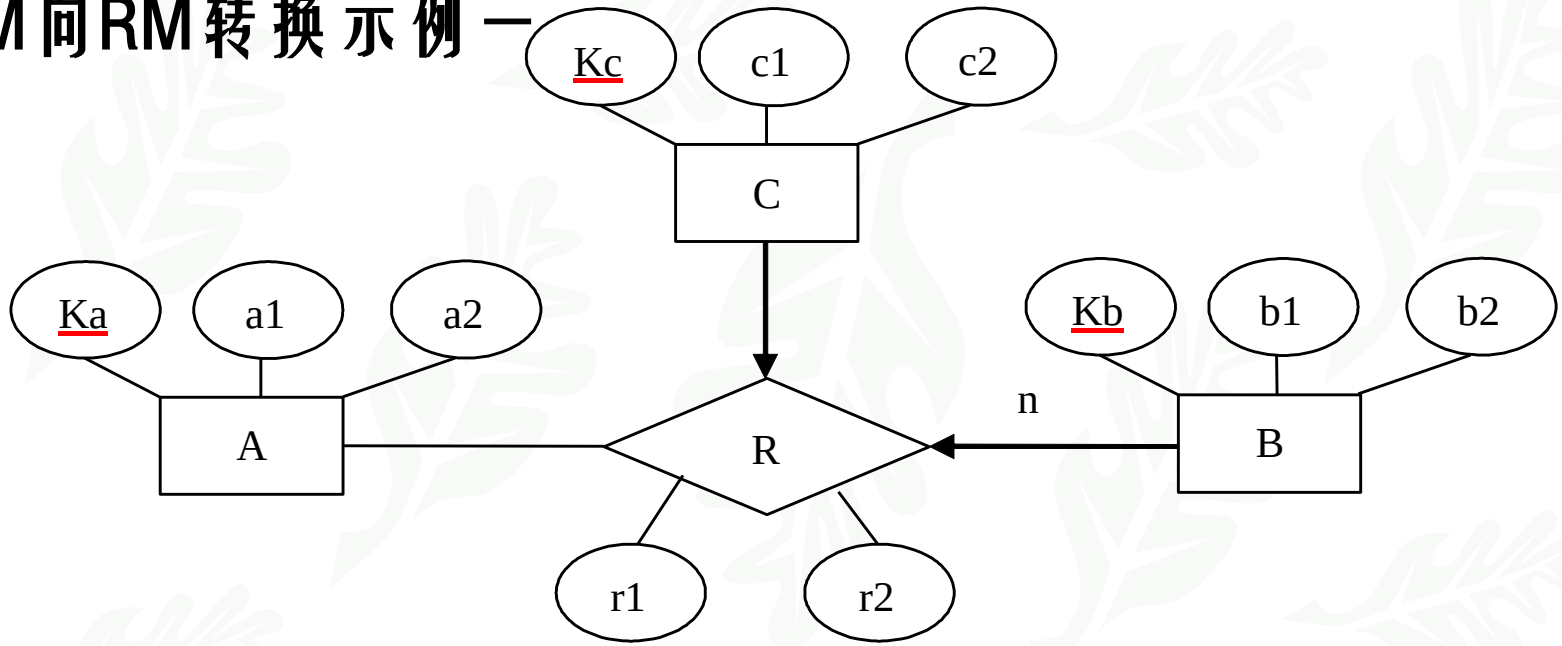


结果为（联系的属性只能向B实体所转换的表移动）：

A (Ka, a1, a2), Ka为主键

B (Kb, b1, b2, r1, r2, Ka), Kb为主键, Ka为外键

（5）ERM向RM转换示例一



转换结果为：

A (Ka, a1, a2), Ka为主键

B (Kb, b1, b2), Kb为主键

C (Kc, c1, c2), Kc为主键

R (Kb, Ka, Kc, r1, r2), 设Kb为主键，则Kc为候选键，Ka, Kb和Kc均为外键

（6）ERM到关系模型转换总结之一

实体 ⇨ 表（实体属性→表的列，实体主键→表的主键）

联系 ⇨

(1) 如联系带有描述属性，则转换为表
（联系的描述属性与涉及的实体键→表的列，联系的键→表的主键）

(2) 如联系不带有描述属性

① 1:1 和 1:n 的联系，不转换为表
（联系涉及的实体的键拷贝到由带有键约束的实体转换的表中）

② m:n 的联系，转换为表
（联系涉及的实体键→表的列，联系的键→表的主键）

（7）ERM到关系模型转换总结之二

实体 ⇨ 表（实体属性→表的列，实体主键→表的主键）

联系 ⇨

(1) m:n 的联系，转换为表

（联系的描述属性与涉及的实体键→表的列，关联实体的主键共同组成表的主键）

(2) 1:1 和 1:n 的联系

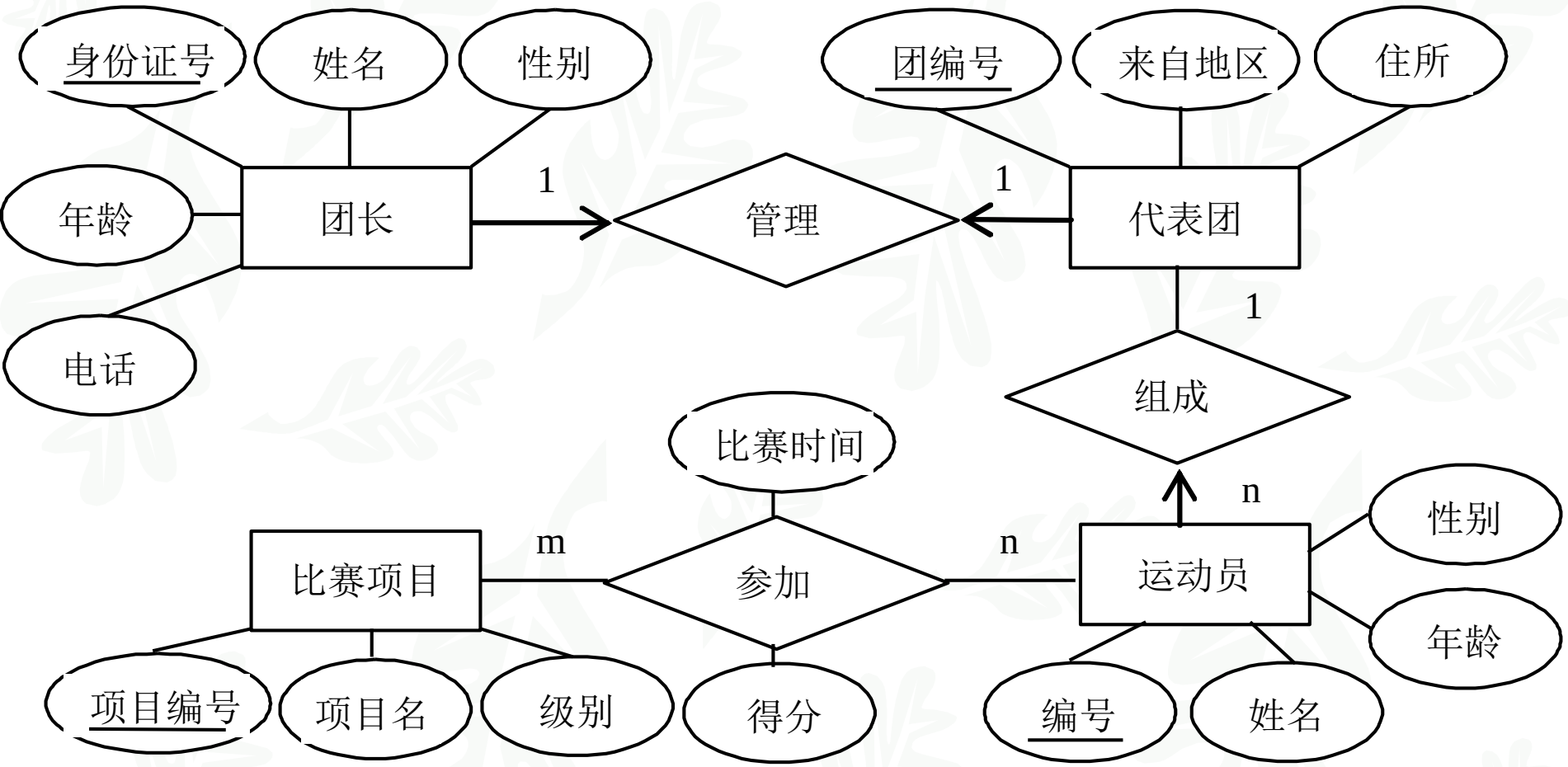
①如联系不带描述属性，不转换为表

（联系涉及的实体的键拷贝到由带有键约束的实体转换的表中）

②如联系带有描述属性，则转换为表

（联系的描述属性与涉及的实体键→表的列，联系的键→表的主键）

（8）ERM向RM转换示例二



转换结果为：

团长（身份证号，姓名，性别，年龄，电话），身份证号为主键。

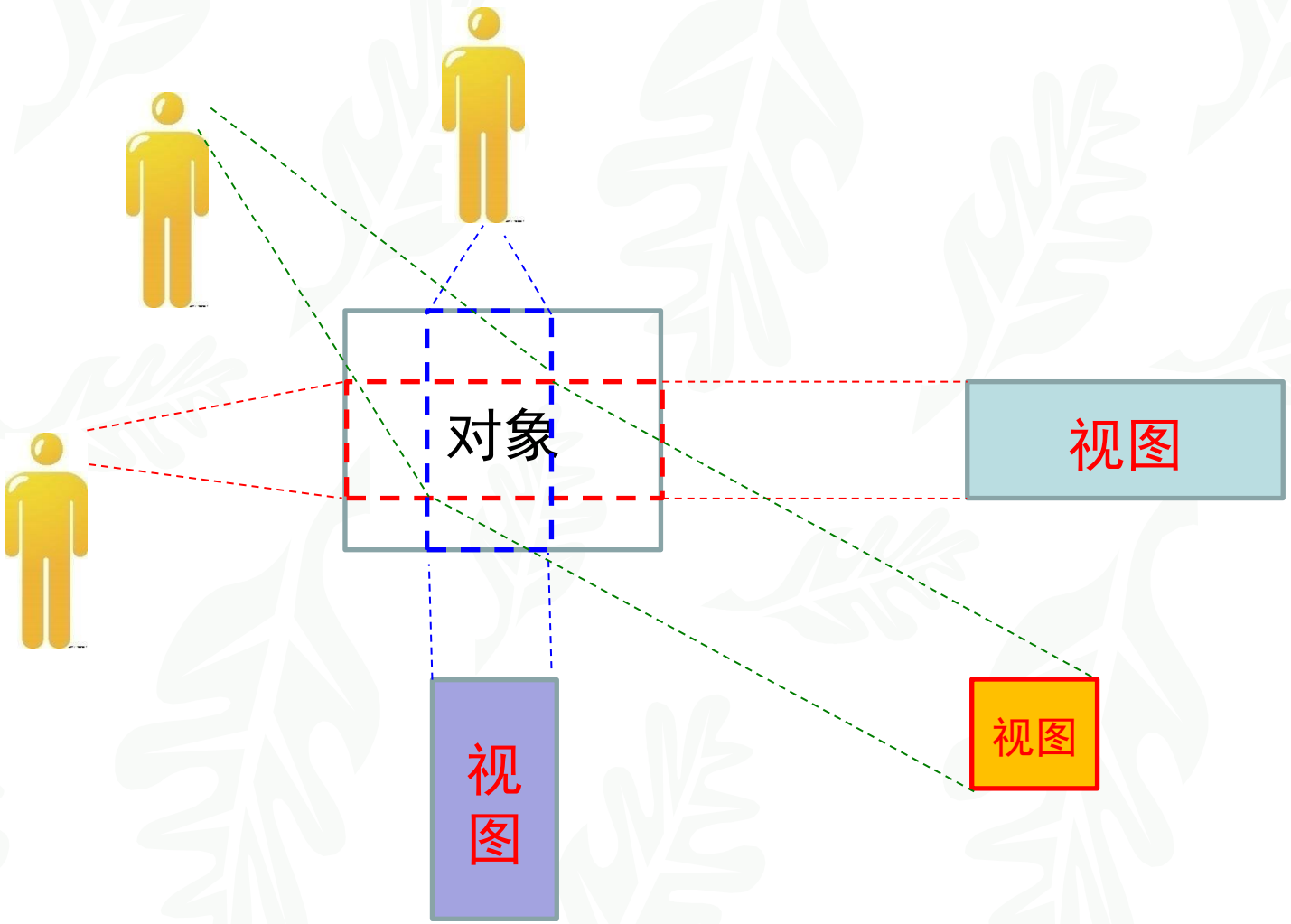
代表团（团编号，来自地区，住所，身份证号），团编号为主键，身份证号既是候选键亦为外键。

运动员（编号，姓名，性别，年龄，团编号），编号为主键，团编号为外键。

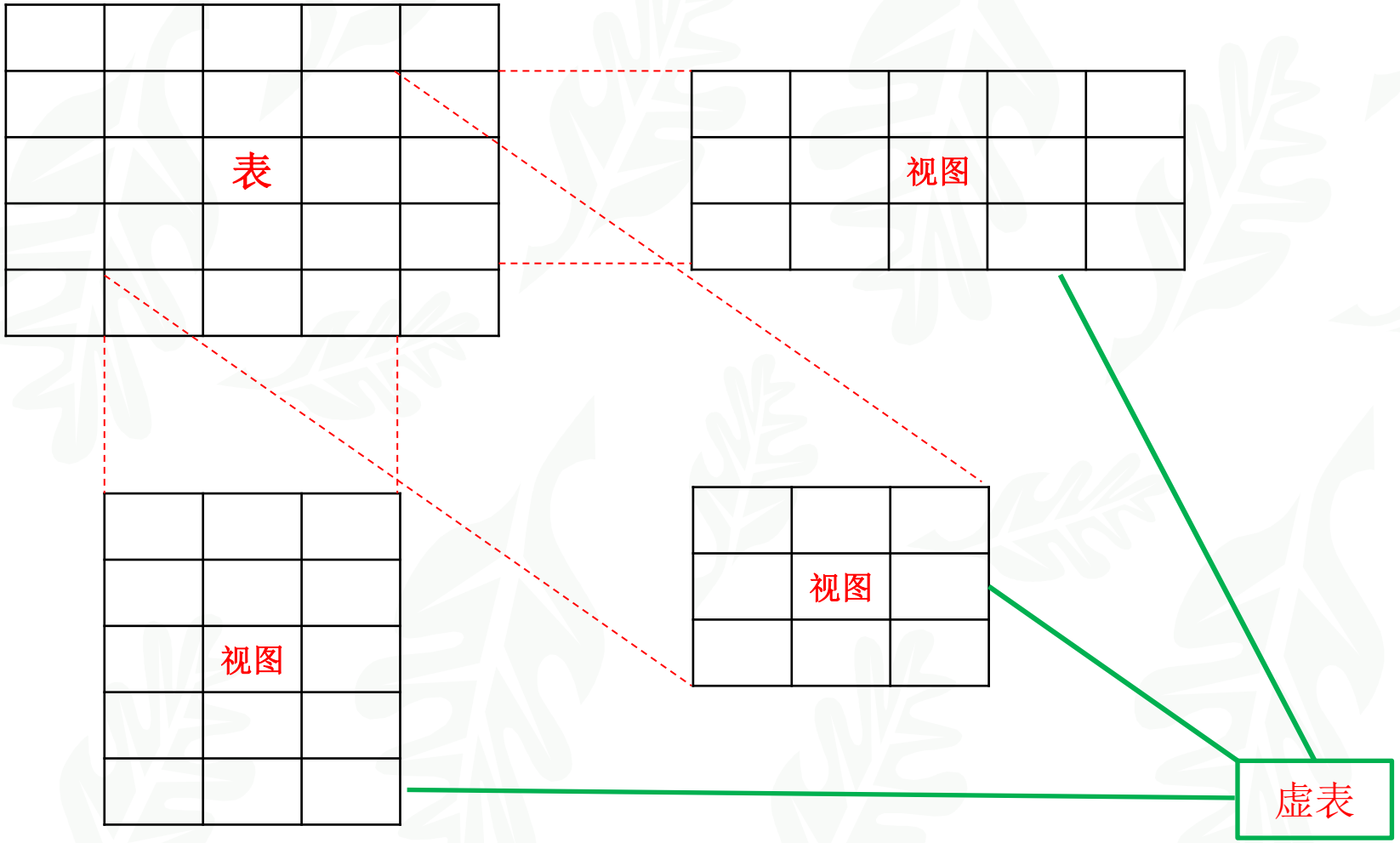
比赛项目（项目编号，项目名，级别），项目编号为主键。

参加（编号，项目编号，比赛时间，得分），（编号，项目编号）为主键，编号和项目编号分别为外键。

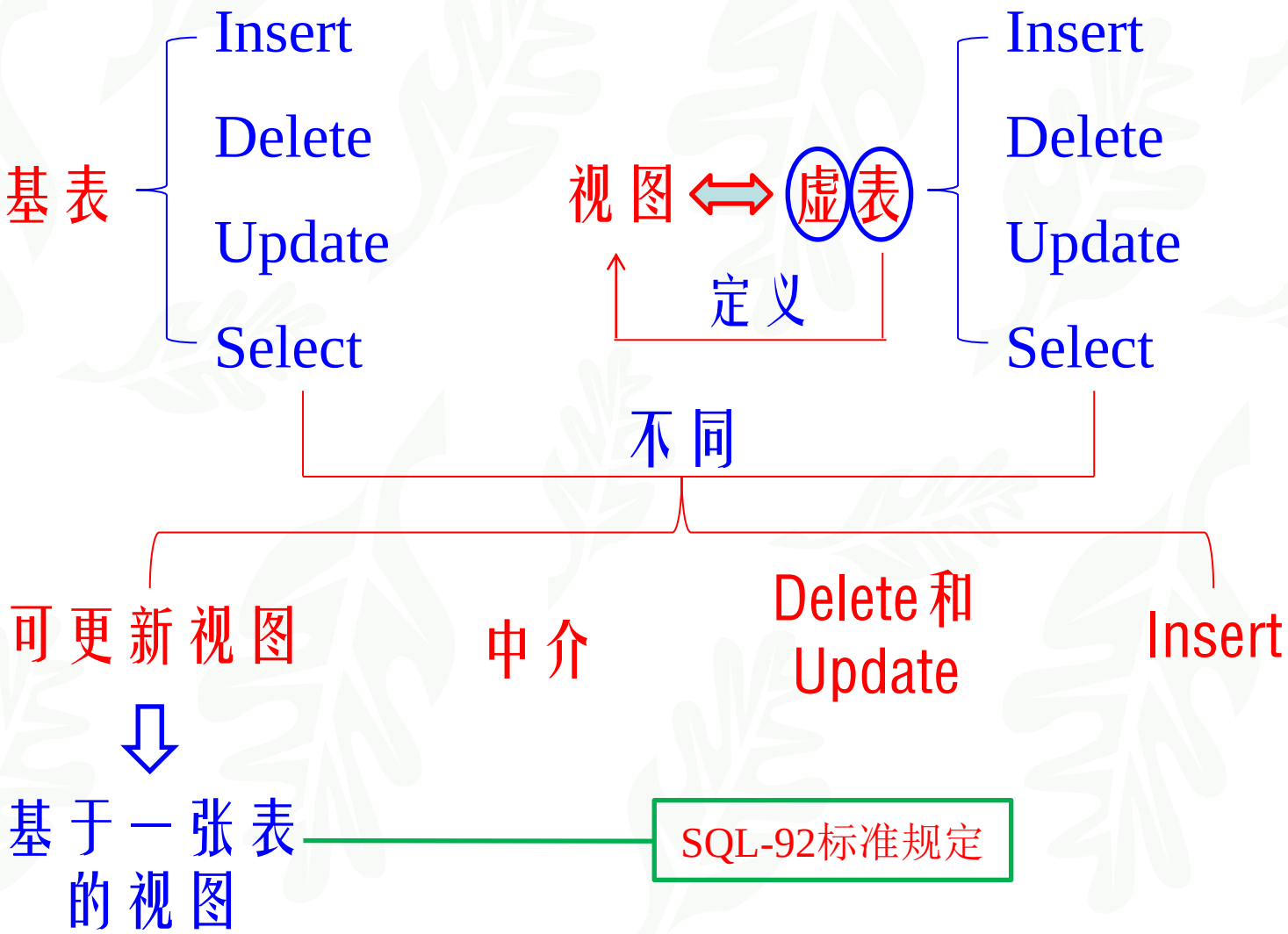
7. Introduction to View



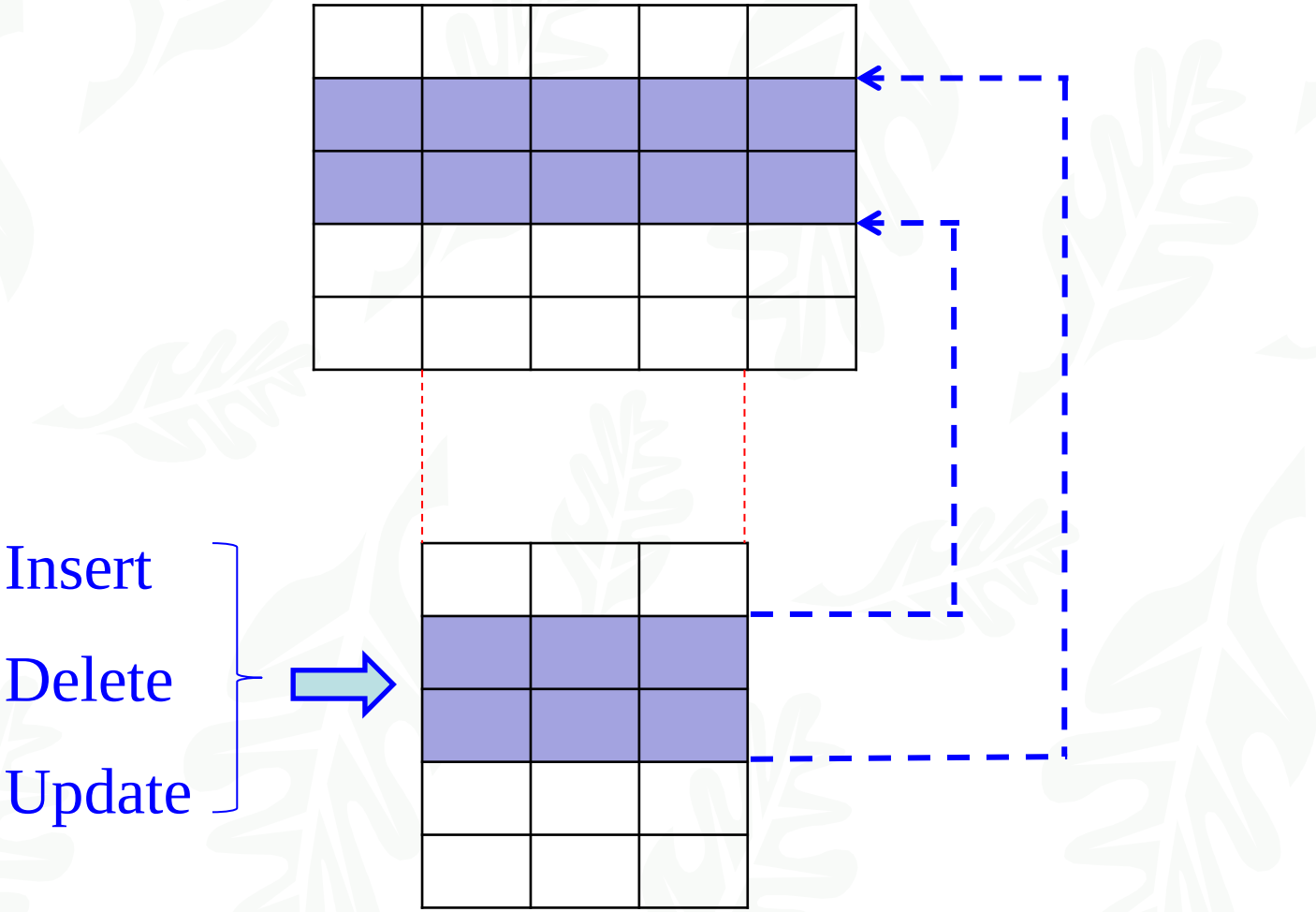
将“对象”换成“表”



基本概念：视图也是一张表，但其不存储数据，其数据是从基表查询出来，故有时称其为“虚表”。



中介作用：通过视图向基表更新数据



安全性支持：由于视图是建立在基表之上，因此对安全性可提供一定的支持。

表与视图之关联：既然视图是一张“虚表”，故而可象基表一样用于定义新的查询或视图，也可以通过视图来更新基表。但在更新视图时，应记住视图与表的区别及对视图更新的限制。

视图的创建：

```
CREATE VIEW  视图名 [列名表]
AS
    SELECT查询
```

示例：

```
CREATE VIEW ca_authors AS
SELECT au_id, au_lname, au_fname, phone, state, contract
FROM  authors
WHERE  state='CA'
```

视图查看：sp_help 视图名

查看视图的创建过程：sp_helptext 视图名

删除视图定义：DROP VIEW 视图名

（1）Update and Delete

SQL-92允许对基于一张表的视图进行更新。可进行更新的视图叫可更新视图。对这种视图的更新即是对其基表更新，即通过修改/删除视图中的行来修改/删除基表中对应的行。

示例：

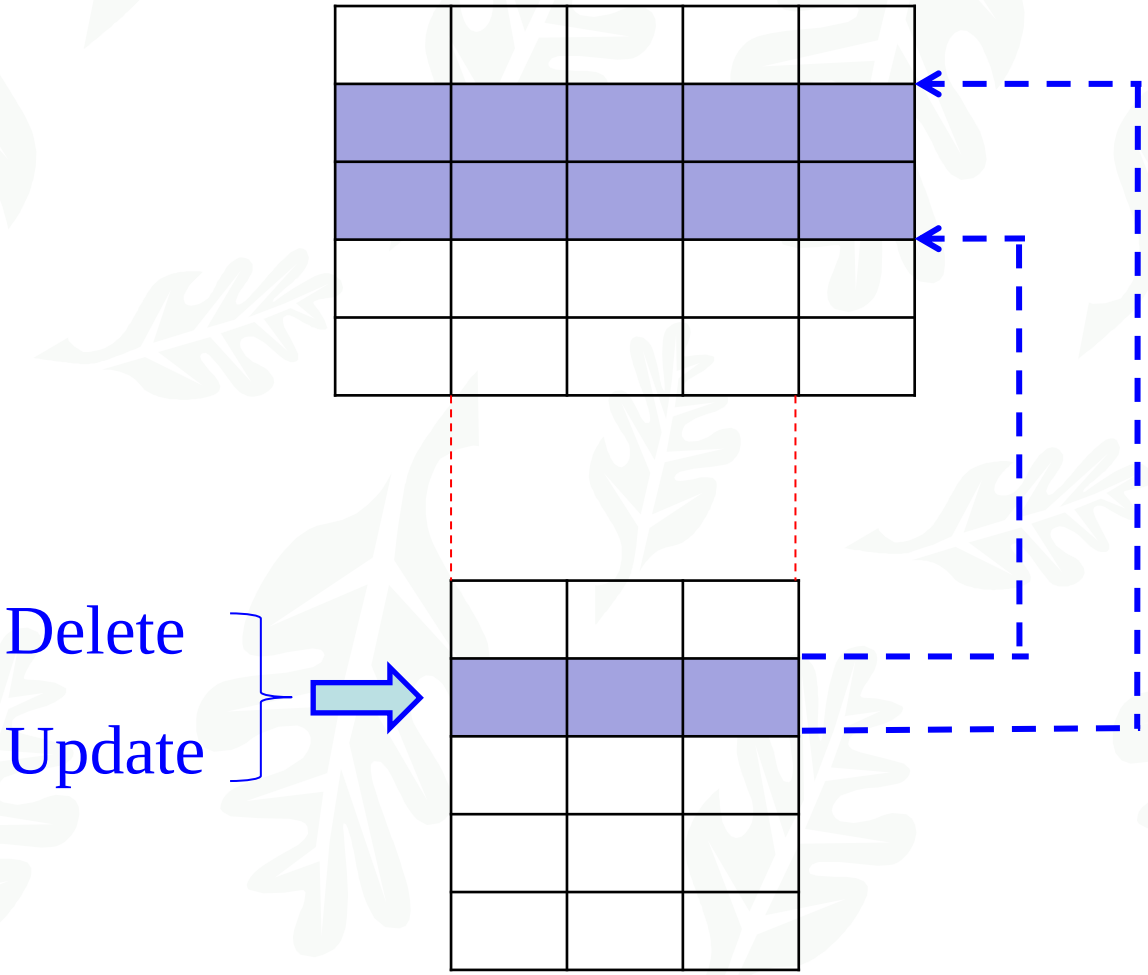
```
UPDATE ca_authors   SET phone='888 496-7223'
WHERE  au_id='172-32-1176'
```

验证：

① SELECT * from au_authors　查看视图数据

② SELECT * from authors　　查看基表中的数据

注意：SQL-92规定：如果视图中没有主键，那么修改/删除视图中的一行有可能会修改/删除基表中的多行。但具体产品如SQL Server，只修改/删除基表中第一个对应的行。



示例：

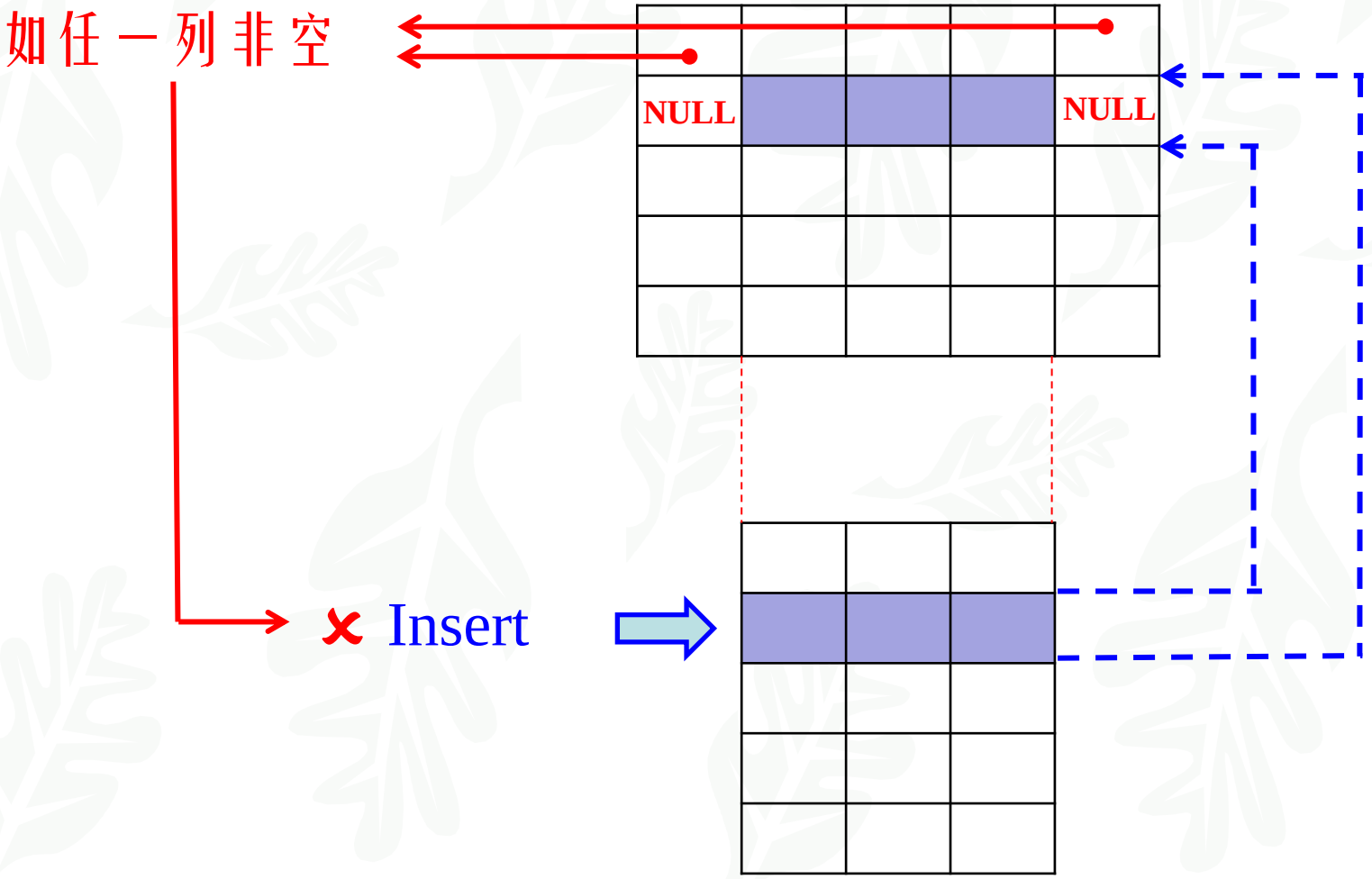
① CREATE VIEW ca_authors1 AS
SELECT au_lname, au_fname, phone, state, contract
FROM authors WHERE state='CA'

② UPDATE ca_authors1
SET au_lname='Greene' WHERE au_lname='Green'

③ UPDATE ca_authors1
SET phone='888 888–8888' WHERE au_lname='Greene'

（2）Insert

对视图插入一行也会在基表中插入一行。对表中没有出现在视图中的列，以NULL代替。



示例：

① CREATE VIEW ca_authors2 AS
SELECT au_id, au_lname, au_fname, phone, state,contract
FROM authors
WHERE state='CA'

② INSERT INTO ca_authors2
VALUES ('888–88–8888','Tao','Hongcai',
'028 888–8888', 'CA',1)

③ SELECT * FROM ca_authors2
SELECT * FROM authors

注意：如果视图中没有包含表中所有非空列（包括主键），那么对视图的插入将被拒绝。

示例：

```
INSERT INTO ca_authors1
VALUES (‘Wang',‘Wu','028 666–6666',‘CA',1)
```

② 错误信息

```
Cannot insert the value NULL into column 'au_id', table
'test1.dbo.authors'; column does not allow nulls.
INSERT fails. The statement has been terminated.
```

特别注意：对视图的插入，有可能数据插入到基表了，但视图中却没有出现。SQL–92缺省行为是允许这种插入，如在视图定义中加入With Check Option可禁止此情况的发生。

示例：

①

```
INSERT INTO ca_authors2
VALUES (‘666–66–6688',‘Zhang',‘Shan',
                '028 666–6688',‘UT',1)
```

② SELECT * FROM ca_authors2
SELECT * FROM authors

③ DROP VIEW ca_authors2

④ CREATE VIEW ca_authors2 AS
SELECT au_id, au_lname, au_fname,
phone, state, contract
FROM authors
WHERE state='CA'
WITH CHECK OPTION

⑤ INSERT INTO ca_authors2
VALUES ('777-77-6688','Zhang','Shan',
'028 666-6688','UT',1)

错误信息: The attempted insert or update failed because the target view either specifies WITH CHECK OPTION or spans a view that specifies WITH CHECK OPTION and one or more rows resulting from the operation did not qualify under the CHECK OPTION constraint. The statement has been terminated.

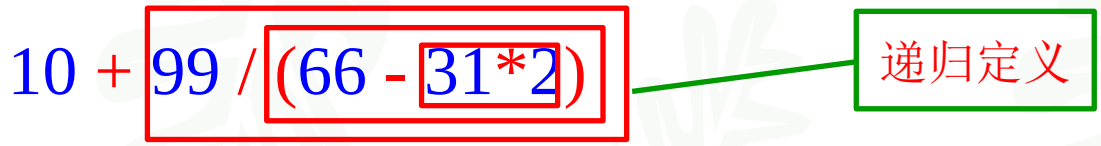
3.2 关系代数

关系代数(Relational Algebra)：与关系模型有关的**查询语言**。当然，与SQL查询语言一样，它们不仅支持查询，还支持数据的插入、修改和删除操作。

关系代数中的查询着重“过程”，故由“操作符”集合组成，是“过程化”的查询。

关系代数与初等代数的着重过程类比：

初等代数：由运算符(自含运算优先级)与运算数构成的表达式。



运算符(初等代数)——操作符(关系代数)

运算数(初等代数)——关系(关系代数)

基本特性：每个“操作符”（Operator）接受一个或二个关系实例（集合值）作为其操作数，并返回一个关系实例（集合值）作为结果。

关系操作符：一元(unary)和二元(binary)。

关系代数表达式：可递归定义。

注意：

① 查询的输入输出均是“关系”，即输入为关系，结果输出仍为关系；

{<11,李峰,20,3>, <52,苏远征,21,3>, <66,陈胜,23,4>}

② 关系查询时，既可用字段名，也可用字段位置来表示字段。

基本代数操作符：

SELECTION(选择)　　PROJECTION(投影)

UNION(并)　　INTERSECT (交)

DIFFERENCE(差)　　CROSS-PRODUCT(积)

附加操作符：基于基本操作符定义。

示例关系模式及实例：

Students (sid(学号)：integer, sname(姓名)：string,
age(年龄)：real, grade(年级)：integer)

Courses (cid(课程号)： integer, cname(课程名)： string,
credit(学分)：integer)

Enrollments (sid：integer, cid： integer, score(成绩)：real)

S1

sid	sname	age	grade
8	何大明	19	2
11	李 峰	20	3
35	陈 胜	21	4

S2

sid	sname	age	grade
6	张大卫	18	1
11	李 峰	20	3
52	苏远征	21	3
66	陈 胜	23	4

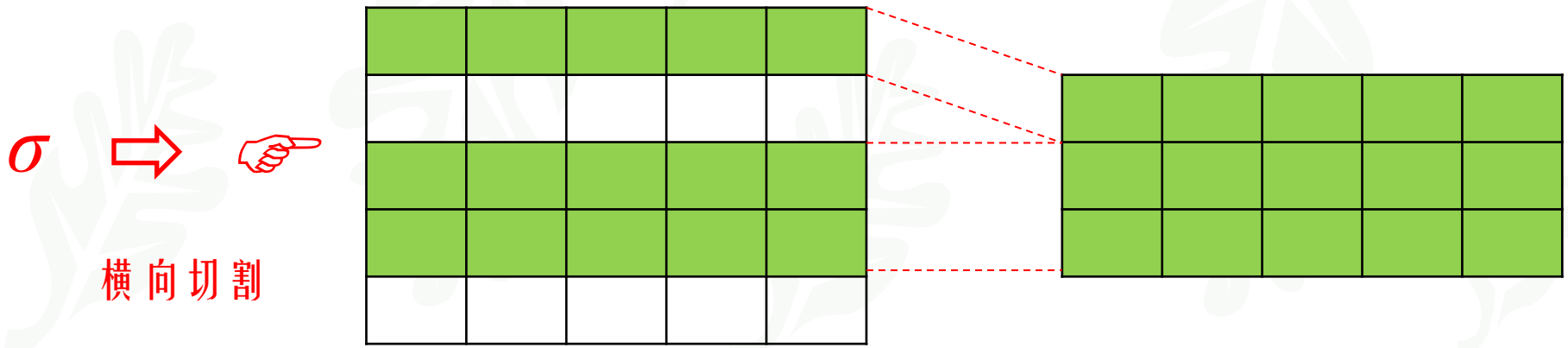
E1

sid	cid	score
8	101	91
66	108	80

1. SELECTION AND PROJECTION

共同点：均为一元关系操作符。

选择：从关系实例中选择出满足条件(Condition)的行。操作符：σ，条件以下标形式表示：σ_c



条件表达式中的比较操作符：＞，＞=，＜，＜=，＝，≠。

示例：$\sigma_{grade>=3}(S2)$

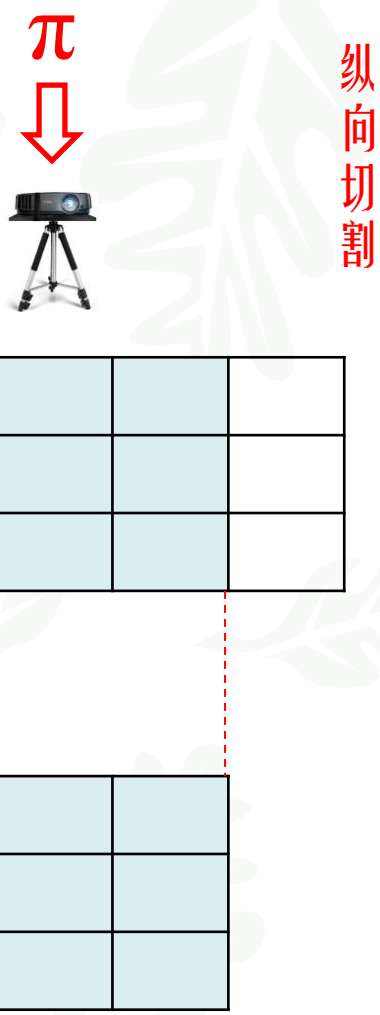
查询结果：

{<11,李峰,20,3>, <52,苏远征,21,3>, <66,陈胜,23,4>}

相应的SQL查询描述：

```
SELECT *
FROM S2
WHERE grade >= 3
```

投影：从关系实例中抽出所需的一列或多列。操作符：π，投影出的列以下标形式列出：$\pi_{a1,a2,\ldots}$。



示例： $\pi_{sname,grade}(S2)$

查询结果：

{<张大卫，1>,<李峰，3>,<苏远征，3>,<陈胜，4>}

相应的SQL查询描述：

```
SELECT  sname, grade
FROM  S2
```

注意：

① 关系代数中，如结果有重复元组，则假设去掉重复元组；

② 在实际DBMS中，由于去掉重复元组开销较大，故保留重复元组；

③ 由于关系代数的结果仍是关系实例，故其结果还可再作为操作符的操作数。

示例： $\pi_{sname,grade}(\sigma_{grade>=3}(S2))$

查询结果：

{<李峰，3>,<苏远征，3>,<陈胜，4>}

相应的SQL查询描述：

```
SELECT  sname, grade
FROM  S2
WHERE  grade >= 3
```

说明：

上述表达式的效果等同于：纵横向切割。

由上可知，关系代数中的选择、投影操作，基本上分别与SQL查询语句SELECT中的WHERE子句和SELECT子句对应。

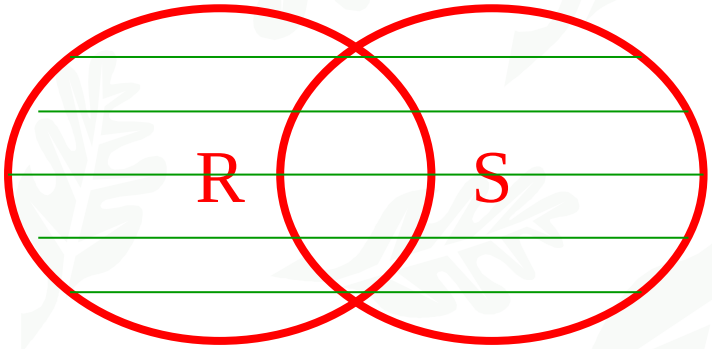
2. 集合运算（Set Operations）

集合运算包括： UNION(并)、INTERSECT（交)、CROSS-PRODUCT(积)、DIFFERENCE(差)。

(1) UNION(并)：R∪S

概念：包含R和S中的所有元组，要求R和S兼容(列数相同、对应列的数据类型兼容)，结果模式与R相同。

R∪S



何谓数据类型兼容?

系统可隐式转换的类型：int, float, …

须用类型转换函数显式转换的类型则不兼容：atoi(), itoa(), …

示例： S1∪S2

查询结果：{<8，何大明，19，2>,<11，李峰，20，3>,
<35，陈 胜，21，4>,<6，张大卫，18，1>,
<52，苏远征，21，3>,<66，陈胜，23，4>}

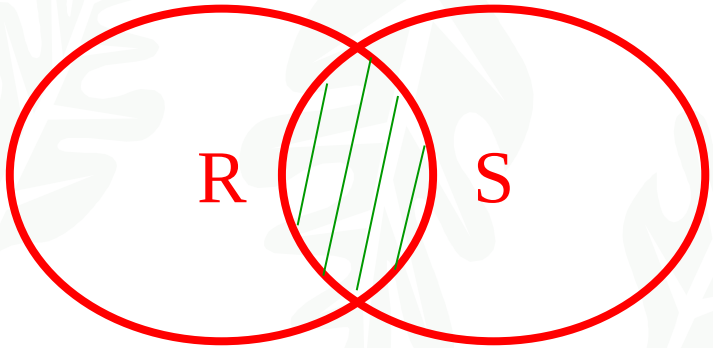
相应的SQL查询描述：

```
SELECT * FROM S1
UNION
SELECT * FROM S2
```

(2) INTERSECT (交)：R∩S

概念：包含R、S中相同的元组，R、S须兼容，结果模式与R相同。

R∩S



示例： **S1∩S2**

查询结果：{<11，李峰，20，3>}

相应的SQL查询描述：

```
SELECT * FROM S1
INTERSECT
SELECT * FROM S2
```

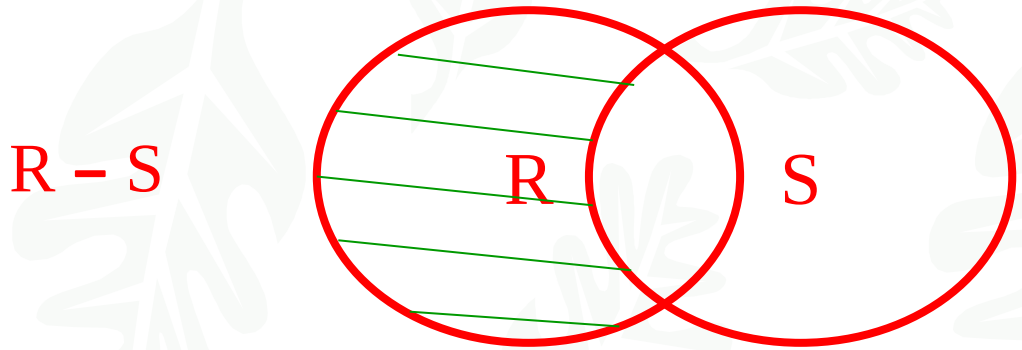
或：

```
SELECT * FROM S1
WHERE S1.sid IN (SELECT S2.sid FROM S2)
```

嵌套子查询

(3) SET-DIFFERENCE(差)：R－S

概念：包含在R中而不在S中的元组，R、S兼容，结果模式与R相同。



示例：S1－S2

查询结果：{<8，何大明，19，2>,<35，陈胜，21，4>}

相应的SQL查询描述：

```
SELECT * FROM S1
EXCEPT
SELECT * FROM S2
```

Oracle用Minus

或：

```
SELECT * FROM S1
WHERE S1.sid NOT IN (
    SELECT S2.sid FROM S2)
```

说明：由于R∩S=R－(R－S)，故严格说来R∩S不是基本的关系代数操作符。

(4) CROSS-PRODUCT(积)：R×S

概念：R的每一行与S的所有行组合，结果包含R和S中所有列。对相同的列名，则用位置表示。也叫“笛卡尔乘积”。

假如：R为m行n列的表，S为p行k列的表，则R×S的结果为$m\times p$行$n+k$列的表。

示例：S1×E1

查询结果：

真正有用的

S1×E1

(sid)	sname	age	grade	(sid)	cid	score
8	何大明	19	2	8	101	91
8	何大明	19	2	66	108	80
11	李　峰	20	3	8	101	91
11	李　峰	20	3	66	108	80
35	陈　胜	21	4	8	101	91
35	陈　胜	21	4	66	108	80

无用的

优点：可将任意两张表关联，甚至也可以与自身关联。

缺点：会产生大量无用结果。

相应的SQL查询描述：

```
SELECT  S1.sid, sname, age, grade,
        E1.sid, cid, score
FROM S1，E1
```

说明：SQL查询语句SELECT中的FROM子句相当于关系代数中的笛卡尔乘积；

3. Renaming

为解决R×S中产生的名字冲突，而引入“改名(ρ)”操作符。

表达式：ρ（R(F)，E）

说明：

①对任意代数表达式E (Expression)，ρ（R(F)，E）返回一个新的关系实例R，其元组与E相同，模式与E也相同，但某些列被改名。

②F的形式为：旧名→新名，或位置→新名。

③改名有两个作用：一是有名字冲突时可改名；二是作为表可临时保存表达式E的结果，表名为R。

示例： ρ（D(1→sid1, 5→sid2)，S1×E1）

查询结果：

D

sid1	sname	age	grade	sid2	cid	score
8	何大明	19	2	8	101	91
8	何大明	19	2	66	108	80
11	李　峰	20	3	8	101	91
11	李　峰	20	3	66	108	80
35	陈　胜	21	4	8	101	91
35	陈　胜	21	4	66	108	80

改名操作结果示意图

4. Joins

概念：关系与关系的连接。可定义为R×S后跟选择。

种类：条件连接、等连接、自然连接、外连接。

(1) Condition Joins(条件连接)

概念：$R \bowtie_c S = \sigma_c(R \times S)$

说明：条件c会用到R和S的列，如R.name，R.i(位置)。

示例：$S1 \bowtie_{S1.sid<E1.sid} E1$

查询结果：

(sid)	sname	age	grade	(sid)	cid	score
8	何大明	19	2	66	108	80
11	李　峰	20	3	66	108	80
35	陈　胜	21	4	66	108	80

条件连接示意图

相应的SQL查询描述：

```
SELECT  S1.sid, sname, age, grade,
        E1.sid, cid, score
FROM  S1  CROSS  JOIN  E1
WHERE S1.Sid < E1.Sid
```

或：

```
SELECT  S1.sid, sname, age, grade,
        E1.sid, cid, score
FROM  S1  ,  E1
WHERE S1.Sid < E1.Sid
```

说明：FROM S1 CROSS JOIN E1 等效于 FROM S1, E1。

(2)等连接 (Equijoin)

概念：是条件连接的特例，即连接条件由等式组成，如R.name1=S.name2。

说明：由于列相等，因此结果中有重复的列，等连接定义中将此重复列在结果中去掉。

示例：$S1 \bowtie_{S1.sid=E1.sid} E1$

查询结果：

sid	sname	age	grade	cid	score
8	何大明	19	2	101	91

相应的SQL查询描述：

```
SELECT  S1.sid, sname, age, grade, cid, score
FROM  S1，E1
WHERE S1.Sid = E1.Sid
```

(3) Natural Join(自然连接)

概念：是等连接的特例，即：等式中所涉及的列名相同，这时可隐去连接条件，即为：R⋈S。

示例：S1⋈E1

查询结果：同上。

这种关联才是最自然的。

相应的SQL查询描述：

```
SELECT S1.sid, sname, age, grade, cid, score
FROM S1 NATURAL JOIN E1
```

或：

```
SELECT S1.sid, sname, age, grade, cid, score
FROM S1 , E1
WHERE S1.Sid = E1.Sid
```

(4) 外连接 (Outer Joins)

概念：涉及有空值的自然连接，是自然连接的特例。

说明：自然连接是寻找两表中相同列值相等的对应行。外连接除要寻找相同列值相等的对应行之外，还要列出一张表在另一张表中没有相同列值相等的对应行。因无对应，结果中某些列的值显示为NULL值。

外连接的种类：

① 左外连接 ⟕ (LEFT OUTER JOIN)
② 右外连接 ⟖ (RIGHT OUTER JOIN)
③ 全外连接 ⟗ (FULL OUTER JOIN)

种类的理解：因须对应，故要有参照点。以外连接操作符左边的表为参照，即为左外连接；如参照右表，则为右外连接；如既参照左表亦参照右表，则为全外连接。

说明：与外连接对应，前面三种连接为**内连接**(Inner Join)。

左外连接：对于R⋈S，以左表R为准（参照），其每一行与S表所有行匹配，有对应列值相等的行即放入结果；如R中某行与S表任一行无对应列值相等，则结果中某些列以NULL表示之。

相应的SQL查询描述：

```
SELECT S1.sid, cid
FROM S1 NATUAL LEFT OUTER JOIN E1
```

示例及结果：

S1

sid	sname	age	grade
8	何大明	19	2
11	李　峰	20	3
35	陈　胜	21	4

E1

sid	cid	score
8	101	91
8	106	80
35	106	84

结果：

sid	**cid**
8	101
8	106
11	null
35	106

左外自然连接示意图

右外连接：对于R⋈S，以右表S为准（参照），其每一行与R表所有行匹配，有对应列值相等的行即放入结果；如S中某行与R表任一行无对应列值相等，则结果中某些列以NULL表示之。

相应的SQL查询描述：

```
SELECT E1.sid, cid, sname
FROM S1 NATURAL RIGHT OUTER JOIN E1
```

示例及结果：

S1

sid	sname	age	grade
8	何大明	19	2
11	李 峰	20	3
35	陈 胜	21	4

E1

sid	cid	score
8	101	91
35	106	84
66	119	88
66	101	99

结果：

sid	cid	sname
8	101	何大明
35	106	陈 胜
66	119	null
66	101	null

右外自然连接示意图

全外连接：对于R ⋈ S，先做左外，再做右外，最后做Union。

相应的SQL查询描述：

```
SELECT S1.sid, cid, S1.sname
FROM S1 NATURAL FULL OUTER JOIN E1
```

示例及结果：

S1

sid	sname	age	grade
8	何大明	19	2
11	李 峰	20	3
35	陈 胜	21	4

E1

sid	cid	score
8	101	91
35	106	84
66	119	88
8	106	80

结果：

sid	cid	**sname**
8	101	何大明
11	null	李 峰
35	106	陈 胜
66	119	null
8	106	何大明

全外自然连接示意图

6. 关系代数查询的若干示例

所用实例：如下图。

S3

sid	sname	age	grade
6	张大卫	18	1
8	何大明	19	2
11	李　峰	20	3
35	陈　胜	21	4
52	苏远征	21	3
66	陈　胜	23	4

C1

cid	cname	credit
101	程序设计	2
102	程序设计	4
103	数据结构	4
104	数 据 库	3

E2

sid	cid	score
8	101	91
8	102	93
8	103	86
8	104	88
11	102	80
11	104	75
35	101	81
35	102	85
66	103	83

实例示意图

Q1： 查询选修103课程的学生名的关系代数表达式。

方法：

①最后结果哪几列；→学生名。

②涉及哪几张表；→学生表，选课表。

③条件与查询思路。→先从选课表选择出选修103课程的学生，再与学生表自然连接，最后投影出学生名。

表达式：$\pi_{sname}((\sigma_{cid=103} E2) \bowtie S3)$

或：$\pi_{sname}(\sigma_{cid=103}(E2 \bowtie S3))$

模板：$\pi_{列1, 列2, \ldots}(\sigma_{条件1 \wedge 条件2 \wedge \ldots}(T1 \bowtie T2 \bowtie \ldots))$

①哪几列

③条件

②哪几张表，多于2张表时须注意顺序

查询结果：{<何大明>,<陈胜>}

亦可分步写：须使用改名操作的保存结果的作用

① ρ (Temp1, $\sigma_{cid=103}$ (E2))

② ρ (Temp2, Temp1$\bowtie$ S3)

③ π_{sname}(Temp2)

说明：

(1) 查询表达式不唯一，只要结果相同即可。

(2) 第一个表达式比第二个更好，因为中间结果更少。此即查询优化。优化的核心思想即是让参与运算的表尽量小；优化方法是对参与运算的表做选择(横向切割)和投影(纵向切割)。

优化：$\pi_{sname}((\pi_{sid}(\sigma_{cid=103}(E2)))\bowtie(\pi_{sid,\ sname}(S3)))$

横向和纵向切割

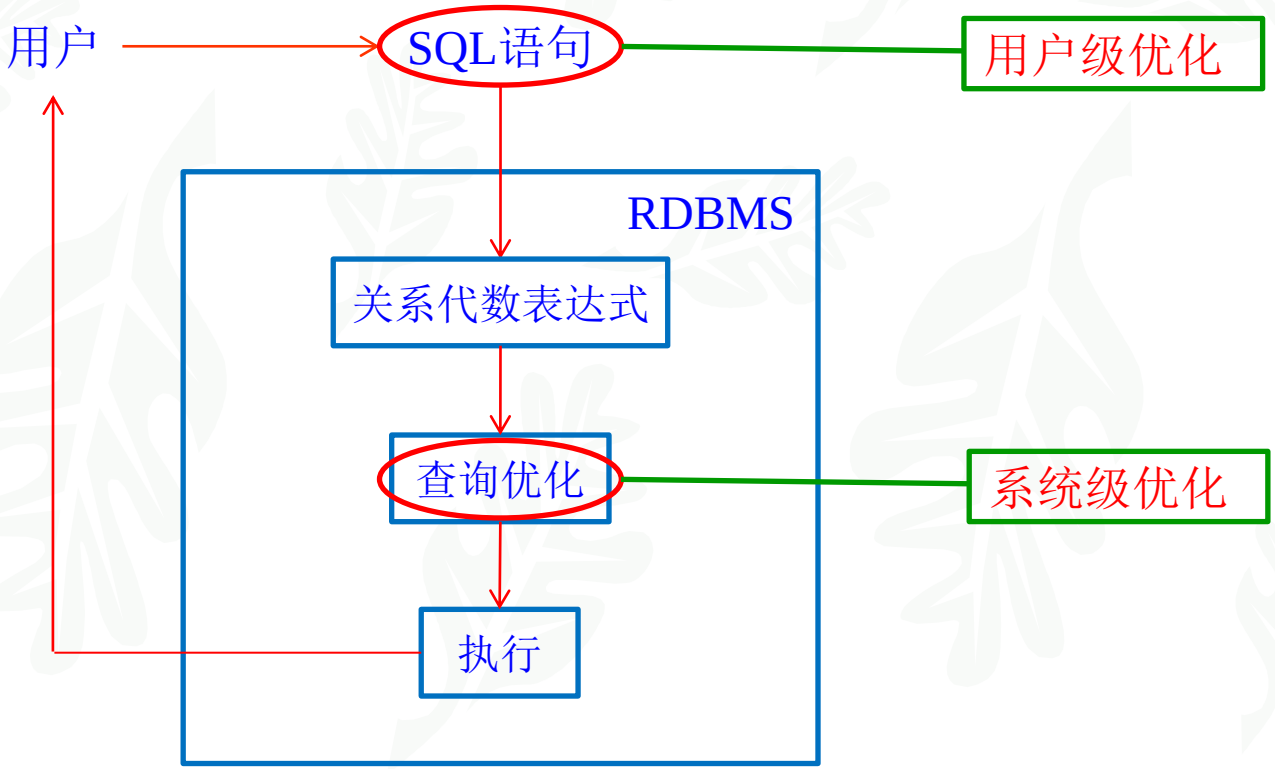
纵向切割

SQL查询：

```
SELECT sname
FROM S3, E2
WHERE S3.sid=E2.sid AND cid=103
```

- SELECT sname：①哪几列；对应投影
- FROM S3, E2：②哪几张表，多于2张表时须注意顺序；对应乘积
- WHERE S3.sid=E2.sid AND cid=103：③条件；对应选择

SQL与关系代数的关系



Q2： 查询选修学分为3的学生名的代数表达式。

表达式：$\pi_{sname}((\sigma_{credit=3}(C1 \bowtie E2 \bowtie S3)))$

或：$\pi_{sname}(\pi_{sid}((\sigma_{credit=3} C1) \bowtie E2) \bowtie S3)$

查询结果：{何大明,李峰}

SQL查询：

```
SELECT sname
FROM S3, E2, C1
WHERE S3.sid=E2.sid AND E2.cid=C1.cid AND C1.credit=3
```

Q3： 学生“李峰”所选课程的学分的代数表达式。

表达式：$\pi_{credit}((\sigma_{sname="李峰"}(S3 \bowtie E2 \bowtie C1)))$

查询结果：{4, 3}

SQL查询：

```
SELECT credit
FROM S3, E2, C1
WHERE S3.sid=E2.sid AND E2.cid=C1.cid AND
        S3.sname='李峰'
```

Q4： 至少选修了一门课程的学生的名字的代数表达式。

表达式：$\pi_{sname}(S3 \bowtie E2)$

查询结果：{何大明，李峰，陈胜}

SQL查询：

```
SELECT sname
FROM S3, E2
WHERE S3.sid=E2.sid
```

Q5： 选修了学分为3或为4课程的学生名字的代数表达式。

表达式：

$\rho(TempCourses, (\sigma_{credit=3} C1) \cup (\sigma_{credit=4} C1))$

$\pi_{sname}(TempCourses \bowtie E2 \bowtie S3)$

或：

$\rho(TempCourses, \sigma_{credit=3 \vee credit=4} C1)$

$\pi_{sname}(TempCourses \bowtie E2 \bowtie S3)$

查询结果：{何大明，李峰，陈胜}

SQL查询：

```
SELECT sname
FROM S3, E2, C1
WHERE S3.sid=E2.sid AND E2.cid=C1.cid AND
        (C1.credit=3 OR C1.credit=4)
```

或：

```
SELECT S31.sname
FROM S3 S31, E2 E21, C1 C11
WHERE S31.sid=E21.sid AND E21.cid=C11.cid AND
        C11.credit=3
UNION
SELECT S32.sname
FROM S3 S32, E2 E22, C1 C12
WHERE S32.sid=E22.sid AND E22.cid=C12.cid AND
        C12.credit=4
```

别名：①长名→短名，便于书写
②表与自身乘积，便于区分

Q6：选修了学分为3和4课程的学生名字的代数表达式。

表达式：

ρ (Temp3, π_{sid} (($\sigma_{credit=3}$ (C1 $\bowtie$ E2))))

ρ (Temp4, π_{sid} (($\sigma_{credit=4}$ (C1 $\bowtie$ E2))))

π_{sname}((Temp3 $\cap$ Temp4) $\bowtie$ S3)

查询结果：{何大明，李峰}

SQL查询：

```
SELECT sname
FROM S3 S, E2 E21, C1 C11,
        E2 E22, C1 C12,
WHERE S.sid=E21.sid AND E21.cid=C11.cid AND
      S.sid=E22.sid AND E22.cid=C12.cid AND
      C11.credit=3 AND C12.credit=4)
```

同一张表用了两次，须别名区分

或：

```
SELECT S31.sname
FROM S3 S31, E2 E21, C1 C11
WHERE S31.sid=E21.sid AND E21.cid=C11.cid AND C31.credit=3
      AND S31.sid IN
      ( SELECT S32.sid
        FROM S3 S32, E2 E22, C1 C12
        WHERE S32.sid=E32.sid AND E32.cid=C12.cid
              AND C12.credit=4)
```

Q7：至少选修了两门课程的学生名字的代数表达式。

表达式：

ρ (Temp1, $\pi_{sid, sname, cid}$ (S3 $\bowtie$ E2))

ρ (Temp2(1→sid1, 2→sname1, 3→cid1, 4→sid2,
5→sname2, 6→cid2), Temp1×Temp1)

π_{sname1}($\sigma_{sid1=sid2 \wedge cid1 \neq cid2}$ Temp2)

查询结果：{何大明，李峰，陈胜}

自己与自己做笛卡尔乘积

SQL查询：

```
SELECT sname
FROM S3 S31, E2 E21, S3 S32, E2 E22
WHERE S31.sid=E21.sid AND S32.sid=E22.sid AND
        S31.sid=S32.sid AND E21.cid<>E22.cid)
```

Q8： 没有选修学分为3的课程的、年龄大于20岁的学生学号的代数表达式。

表达式：

$$\pi_{sid}(\sigma_{age>20} S3) - \pi_{sid}((\sigma_{credit=3}(C1 \bowtie E2)))$$

查询结果：{35, 52, 66 }

SQL查询：

```
SELECT sid  FROM S3
 WHERE age>20 AND sid NOT IN
        ( SELECT sid   FROM E2, C1
          WHERE E2.cid=C1.cid AND credit=3)
```